

Files Ka Detailed Analysis

19 ABFYCTA 2026 F.

YOU 19/08/2026, 22:45:29

tum inn dono files ko padho
ek dum detailed me

DEEP RESEARCH 19/08/2026, 22:45:38

Make sure to include `filecite`turn0file0L5-L8 in your response to cite this file, or to surface it as a link.
Replace the placeholder line range with the actual line range from the result that best fits the snippet you want to surface with the citation.

Use `ref_id "turn0file0"` to reference this file. For broad questions or unknown locations, retrieve relevant chunks with `files.search` scoped to this file. Use `files.find` only for exact text, and use `files.read` for a known range or to expand relevant results. Full file size: 3317 file lines.

- [L1] # SHIA-RAG: Complete Project Report
- [L2] ## Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation
- [L4] > **Project Type:** Minor Project (M.Tech)
- [L5] > **Date:** August 2026
- [L6] > **Documents Analyzed:** 6 PDFs + 914-page audit (total ~2M+ characters)
- [L7] > **Audit Findings Fixed:** 75 issues across all 9 layers

- [L11] ## Table of Contents
- [L13] 1. Project Overview
- [L14] 2. Document Inventory & Summary
- [L15] 3. Critical Errors & Fixes
- [L16] 4. Cross-Document Inconsistencies
- [L17] 5. Novelty Contributions
- [L18] - 5.1 Literature Search Methodology
- [L19] - 5.2 Novelty Level Assessment
- [L20] 6. Complete System Architecture
- [L21] 7. Detailed Module Design
- [L22] - 7.1 Layer 0: Core Data Model
- [L23] - 7.2 Layer 1: Input Processing
- [L24] - 7.3 Layer 2: Document Parsing
- [L25] - 7.4 Layer 3: Knowledge Extraction
- [L26] - 7.5 Layer 4: Relation Management
- [L27] - 7.6 Layer 5: SHIA Core
- [L28] - 7.7 Layer 6: Retrieval Engine
- [L29] - 7.8 Layer 7: Generation Engine
- [L30] - 7.9 Layer 8: Operations & Self-Evolution
- [L31] - 7.10 Schema Validation

[L32] - [7.11 Database Indexing](#)
 [L33] - [7.12 Migration Strategy](#)
 [L34] - [7.13 Error Handling](#)
 [L35] - [7.14 Caching Strategy](#)
 [L36] - [7.15 Monitoring & Alerting](#)
 [L37] - [7.16 Multi-Tenancy](#)
 [L38] - [7.17 Async Processing](#)
 [L39] [8. Mathematical Formulations \(Corrected\)](#)
 [L40] [9. Implementation Roadmap](#)
 [L41] [10. Evaluation Framework](#)
 [L42] - [10.4 Statistical Significance](#)
 [L43] - [10.5 Error Analysis](#)
 [L44] - [10.6 Computational Cost](#)
 [L45] [11. Risk Analysis & Mitigation](#)
 [L46] [12. Conclusion](#)

[L50] **## 1. Project Overview**

[L52] **### 1.1 What is SHIA-RAG?**

[L54] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) is a next-generation RAG system that fundamentally changes how information is retrieved for LLM-based question answering.

[L56] **Traditional RAG** splits documents into flat text chunks → embeds them → retrieves top-k similar chunks → feeds to LLM.

[L58] **SHIA-RAG** instead:

[L59] 1. **Extracts knowledge** (concepts, definitions, relations) from documents

[L60] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes (not text chunks)

[L61] 3. **Retrieves knowledge nodes** by traversing the hierarchy (parents, children, siblings)

[L62] 4. **Assembles context** intelligently within a token budget

[L63] 5. **Generates answers** with citation-backed evidence

[L65] **### 1.2 Why It Matters**

[L67] | Problem with Traditional RAG | How SHIA-RAG Solves It |

[L68] |---|---|

[L69] | Chunks break semantic continuity | Knowledge nodes preserve complete concepts |

[L70] | Duplicate/fragmented information | Entity resolution merges synonyms into single nodes |

[L71] | Fixed chunk granularity | Hierarchy allows multi-level retrieval (broad → specific) |

[L72] | Weak multi-hop reasoning | Parent-child paths encode reasoning chains |

[L73] | No cross-document fusion | Knowledge Forest merges concepts across all documents |

[L75] **### 1.3 Core Innovation**

[L77] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "CS → OS → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L81] ## 2. Document Inventory & Summary

[L83] ### 2.1 All 6 PDFs Analyzed

[L85] | # | File Name | Pages | Content Type | What It Contains |

[L86] |---|-----|-----|-----|-----|

[L87] | 1 | first Pdf.pdf | 24 | Original MDS (Academic) | Executive Summary, Problem Statement, Literature Review, System Architecture (13 modules), SHIA Core algorithms (CPG++, PRE, HV, FI, CLD), Evaluation Plan, Implementation Roadmap |

[L88] | 2 | third Pdf.pdf | 10 | Algorithms & Scoring | Detailed scoring formulas for all SHIA modules, pseudocode for 6 algorithms, retrieval policies (RL-based adaptive depth), uncertainty resolution, security considerations |

[L89] | 3 | 4h pdf.pdf | 22 | Formal Specification | Formal mathematical definitions, Energy Minimization objective, C++ interfaces, SHEF evaluation framework, complexity analysis, experimental methodology, reproducibility checklist |

[L90] | 4 | Master Design Specification (SHIA-RAG).pdf | 46 | Production MDS | Complete 8-layer architecture with JSON schemas, SQL/Cypher DB design, OpenAPI contracts, microservice folder structure, Layer-by-layer algorithms with pseudocode |

[L91] | 5 | PDF Comparison Summary.pdf | 236 | Research Comparison | Deep comparison of all PDFs, literature positioning against TreeRAG/HiChunk/RAPTOR/GraphRAG, gap analysis, combined architecture walkthrough |

[L92] | 6 | PDFs discussion and analysis.pdf | 483 | Extended Analysis | Collaborative transcript refining SHIA-RAG, detailed layer-by-layer walkthroughs, critical review identifying gaps, research contribution proposals |

[L94] ### 2.2 How Documents Relate

[L96]

```
[L97] graph TD
[L98]   A["PDF 1: first Pdf<br/>(24p) Original Vision"] --> D["PDF 5: Comparison Summary<br/>(236p) Research Comparison"]
[L99]   B["PDF 2: third Pdf<br/>(10p) Algorithms"] --> D
[L100]  C["PDF 3: 4h pdf<br/>(22p) Formal Spec"] --> D
[L101]  E["PDF 4: Master Design Spec<br/>(46p) Production Blueprint"] --> D
[L102]  D --> F["PDF 6: Discussion & Analysis<br/>(483p) Critical Refinement"]

[L104]  A -->|"Evolved into"| E
[L105]  B -->|"Adds formulas to"| E
[L106]  C -->|"Adds rigor to"| E
[L107]  F -->|"Identifies gaps in"| A
[L108]  F -->|"Identifies gaps in"| E
```

[L113] ## 3. Critical Errors & Fixes

[L115] > [!CAUTION]

[L116] > **24+ issues identified across all 6 PDFs.** Below are the most critical ones with proposed fixes.

[L118] ### 3.1 FATAL: Broken Cycle Detection (PDF 2 — third Pdf.pdf)

[L120] **The Bug:** The DFS cycle detection algorithm traverses the **wrong direction**.

[L122]

```

[L123] # BROKEN (current code)
[L124] - def is_valid(N, P, forest):
[L125] -     stack = [P]                # Start from Parent
[L126] -     while stack:
[L127] -         cur = stack.pop()
[L128] -         for child in forest.children(cur): # Go DOWN from P
[L129] -             if child == N:             # Found N as descendant of P
[L130] -                 return False           # This detects redundancy, NOT cycles
[L131] -                 stack.append(child)
[L132] -     return True

[L134] # FIXED
[L135] + def is_valid(N, P, forest):
[L136] +     """Check: would adding edge P-N create a cycle?
[L137] +     A cycle exists if N is already an ANCESTOR of P.
[L138] +     So traverse UP from P toward roots, looking for N."""
[L139] +     visited = set()
[L140] +     stack = [P]
[L141] +     while stack:
[L142] +         cur = stack.pop()
[L143] +         if cur == N:
[L144] +             return False # N is ancestor of P → cycle!
[L145] +         if cur not in visited:
[L146] +             visited.add(cur)
[L147] +             for parent in forest.parents(cur): # Go UP
[L148] +                 stack.append(parent)
[L149] +     return True

```

[L152] **Impact:** Without this fix, cycles in the Knowledge Forest will go undetected, violating the acyclic invariant and causing infinite loops during traversal.

[L156] ### 3.2 FATAL: No Feedback Loop Between PRE and HV (PDF 3 — 4h pdf.pdf)

[L158] **The Bug:** PRE selects the single best parent → HV validates it. If HV rejects (e.g., cycle detected), the node is **orphaned** with no fallback.

[L160]

```

[L161] # BROKEN (current pipeline)
[L162] - best_parent = PRE.rank(candidates) # Pick ONE best
[L163] - if HV.validate(best_parent, node):
[L164] -     FI.insert(node, best_parent)
[L165] - else:
[L166] -     ??? # Node is lost!

[L168] # FIXED: PRE returns ranked list, HV iterates
[L169] + ranked_parents = PRE.rank_all(candidates) # Return SORTED list
[L170] + inserted = False
[L171] + for parent in ranked_parents:
[L172] +     if HV.validate(parent, node):
[L173] +         FI.insert(node, parent)
[L174] +         inserted = True
[L175] +         break
[L176] + if not inserted:
[L177] +     FI.insert_as_root(node) # Create new tree

```

[L182] ### 3.3 FATAL: Confidence Propagation Without Topological Sort (PDF 2)

[L184] **The Bug:** `propagate_confidence()` iterates over nodes in arbitrary order. A child's confidence depends on its parent's updated confidence, but the parent may not have been processed yet. Additionally, the original formula only uses parent confidence with decay, ignoring the node's own extraction confidence and evidence confidence.

[L186]

```
[L187] # BROKEN
[L188] - for node in forest.nodes(): # Arbitrary order
[L189] -     if node.parents:
[L190] -         node.confidence = avg(p.confidence for p in node.parents) * decay

[L192] # FIXED: Use topological sort (parents always before children)
[L193] # FIXED: Include node's extraction confidence and evidence confidence
[L194] + from collections import deque
+
[L196] + # Confidence weighting factors (must sum to 1.0)
[L197] + ALPHA = 0.4 # Node's own extraction confidence weight
[L198] + BETA = 0.4 # Parent confidence weight
[L199] + GAMMA = 0.2 # Evidence confidence weight
[L200] + DECAY = 0.95 # Per-level decay factor
+
[L202] + def propagate_confidence(forest):
[L203] +     # Kahn's algorithm for topological ordering
[L204] +     in_degree = {n: len(forest.parents(n)) for n in forest.nodes()}
[L205] +     queue = deque([n for n, d in in_degree.items() if d == 0]) # Roots first
[L206] +     while queue:
[L207] +         node = queue.popleft()
[L208] +         if forest.parents(node):
[L209] +             parent_confs = [p.confidence for p in forest.parents(node)]
[L210] +             avg_parent_conf = sum(parent_confs) / len(parent_confs)
[L211] +             evidence_conf = avg([e.confidence for e in node.evidence]) if node.evidence else 0
[L212] +             node.confidence = (
[L213] +                 ALPHA * node.extraction_confidence +
[L214] +                 BETA * avg_parent_conf +
[L215] +                 GAMMA * evidence_conf
[L216] +             ) * DECAY
[L217] +         else:
[L218] +             node.confidence = node.base_confidence # Root nodes (default: 0.5)
[L219] +         for child in forest.children(node):
[L220] +             in_degree[child] -= 1
[L221] +             if in_degree[child] == 0:
[L222] +                 queue.append(child)
```

[L225] > [!NOTE]

[L226] > **base_confidence** is the standardized term used throughout (default: 0.5, configurable). Previously referred to as **baseline_conf** in some PDFs.

[L230] ### 3.4 CRITICAL: RBO Knapsack Ignores Node Dependencies (PDF 6)

[L232] **The Bug:** The Retrieval Budget Optimizer treats nodes as independent items in a 0-1 Knapsack. But in a hierarchy, a child node is meaningless without its parent context.

[L234] > [!WARNING]

[L235] > **Implementation Note:** The algorithm below is a **greedy approximation** for the Precedence-Constrained Tree Knapsack problem, NOT an exact dynamic programming solution. Exact DP for tree knapsack is

$\mathcal{O}(nB)$ where B is the token budget (potentially very large). The greedy approach uses utility-density sorting and provides a practical near-optimal solution.

[L237]

```
[L238] # BROKEN: Standard Knapsack (independent items)
[L239] - nodes.sort(key=lambda n: n.utility / n.token_cost, reverse=True)
[L240] - for n in nodes:
[L241] -     if budget >= n.token_cost:
[L242] -         selected.append(n)
[L243] -         budget -= n.token_cost

[L245] # FIXED: Greedy Approximation for Tree Knapsack (precedence-constrained)
[L246] + def tree_knapsack(root, budget, forest):
[L247] +     """Select nodes maximizing utility, but a child can only be
[L248] +         selected if its parent is also selected.
+
[L250] +         NOTE: This is a greedy approximation (not exact DP).
[L251] +         Children are sorted by utility-density and greedily allocated.
[L252] +         Exact DP is computationally prohibitive for large token budgets."""
[L253] +     dp = {} # Memoization cache
+
[L255] +     def solve(node, remaining_budget):
[L256] +         if (node.id, remaining_budget) in dp:
[L257] +             return dp[(node.id, remaining_budget)]
+
[L259] +         # Option 1: Don't include this node (or any descendants)
[L260] +         best = (0, set())
+
[L262] +         # Option 2: Include this node, then greedily pick children
[L263] +         if remaining_budget >= node.token_cost:
[L264] +             child_budget = remaining_budget - node.token_cost
[L265] +             total_util = node.utility
[L266] +             selected = {node.id}
+
[L268] +             # Greedily allocate remaining budget to children by utility-density
[L269] +             children = sorted(forest.children(node),
[L270] +                             key=lambda c: c.utility / max(c.token_cost, 1),
[L271] +                             reverse=True)
[L272] +             for child in children:
[L273] +                 if child_budget >= child.token_cost:
[L274] +                     cu, cs = solve(child, child_budget)
[L275] +                     total_util += cu
[L276] +                     selected |= cs
[L277] +                     child_budget -= sum(forest.get_node(n).token_cost for n in cs)
+
[L279] +             if total_util > best[0]:
[L280] +                 best = (total_util, selected)
+
[L282] +             dp[(node.id, remaining_budget)] = best
[L283] +             return best
+
[L285] +     return solve(root, budget)
```

[L290] ### 3.5 CRITICAL: Undefined `node_level` for Unplaced Nodes (PDFs 1, 3)

[L292] **The Bug:** CPG++ and PRE use `node_level(q)` to filter candidates by abstraction distance. But `q` is a new, unplaced node — it has no level/depth yet.

[L294] **Fix:** Estimate abstraction level from textual features before placement:

```

[L296] ```python
[L297] def estimate_abstraction_level(node):
[L298] """Multi-factor heuristic for estimating abstraction level of unplaced nodes.
[L299] Returns a score in [0.0, 1.0] where higher = more abstract.

[L301] NOTE: Simple name-length alone is insufficient (e.g., 'A' → abstract? No).
[L302] We combine multiple signals for robustness."""
[L303] score = 0.0

[L305] # Factor 1: Name length (shorter multi-word names tend to be more abstract)
[L306] word_count = len(node.name.split())
[L307] if word_count <= 2:
[L308] score += 0.15 # Short names like "Scheduling", "CPU" are abstract
[L309] elif word_count >= 4:
[L310] score -= 0.05 # Long names like "Advanced Encryption Standard" are specific

[L312] # Factor 2: Number of source documents (more sources = more general)
[L313] score += min(len(node.sources) 0.05, 0.20)

[L315] # Factor 3: Number of sub-concept hints (more sub-concepts = more abstract)
[L316] score += min(len(node.children_hints) 0.05, 0.25)

[L318] # Factor 4: Definition keywords indicating abstraction
[L319] abstract_keywords = ['general', 'category', 'type', 'class', 'family', 'group',
[L320] 'framework', 'paradigm', 'app

```

DEEP RESEARCH 19/08/2026, 22:45:38

Make sure to include `filecite{turn0file1}{L5-L8}` in your response to cite this file, or to surface it as a link.

Replace the placeholder line range with the actual line range from the result that best fits the snippet you want to surface with the citation.

Use `ref_id "turn0file1"` to reference this file. For broad questions or unknown locations, retrieve relevant chunks with `files.search` scoped to this file. Use `files.find` only for exact text, and use `files.read` for a known range or to expand relevant results. Full file size: 1528 file lines.

[L1] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L3] > **Project Type:** Minor Project (M.Tech)

[L4] > **Date:** August 2026

[L5] > **Author:** [Your Name]

[L9] ## Abstract

[L11] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L13] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L15] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), the first benchmark specifically designed for hierarchical RAG quality.

[L17] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L19] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L23] **## Table of Contents**

[L25] 1. [Introduction](#)

[L26] 2. [Literature Review](#)

[L27] 3. [System Architecture](#)

[L28] 4. [Data Model — Layer 0](#)

[L29] 5. [Offline Pipeline — Layers 1-5](#)

[L30] 6. [Online Pipeline — Layers 6-7](#)

[L31] 7. [Operations Layer — Layer 8](#)

[L32] 8. [Novel Contributions](#)

[L33] 9. [Mathematical Framework](#)

[L34] 10. [Implementation](#)

[L35] 11. [Evaluation Framework](#)

[L36] 12. [Risk Analysis](#)

[L37] 13. [Conclusion & Future Work](#)

[L41] **## 1. Introduction**

[L43] **### 1.1 Problem Statement**

[L45] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L47] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L48] 2. Embed each chunk into a vector space

[L49] 3. At query time, retrieve the top-k most similar chunks

[L50] 4. Feed these chunks as context to an LLM for answer generation

[L52] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L54] | Problem | Description | Impact |

[L55] |-----|-----|-----|

[L56] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L57] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L58] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L59] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |

[L60] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L62] ### 1.2 Proposed Solution: SHIA-RAG

[L64] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L67] Traditional RAG:	Document $\rightarrow$ [Chunk ₁ , Chunk ₂ , ..., Chunk _n] $\rightarrow$ Top-k $\rightarrow$ LLM
[L68] SHIA-RAG:	Document $\rightarrow$ Knowledge Forest (concepts + relations) $\rightarrow$ Adaptive Traversal $\rightarrow$ LLM

[L71] The system:

[L72] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L73] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L74] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L75] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L76] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L77] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L79] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science $\rightarrow$ Operating Systems $\rightarrow$ Scheduling $\rightarrow$ Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L81] ### 1.3 Objectives

[L83] 1. Design a complete 8-layer architecture for hierarchical RAG

[L84] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L85] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L86] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L87] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L88] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L90] ### 1.4 Contributions

[L92] This work makes five novel contributions to the RAG literature:

[L94] | # | Contribution | Novelty Level |

[L95] |---|-----|-----|

[L96] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L97] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L98] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L99] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L100] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L104] ## 2. Literature Review

[L106] ### 2.1 Related Work

[L108] ##### 2.1.1 Flat-Chunk RAG Systems

[L110] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Guu et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-reflection, but all operate on flat text chunks with no structural awareness.

[L112] ##### 2.1.2 Structured RAG Systems

[L114] Recent work has begun exploring structured retrieval:

[L116] | System | Approach | Key Limitation |

[L117] |-----|-----|-----|

[L118] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L119] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L120] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L121] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L123] ##### 2.1.3 Knowledge Graph Construction

[L125] Ontology learning and taxonomy induction have a rich history in NLP:

[L126] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L127] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L128] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L130] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**

[L132] ### 2.2 Research Gap

[L135]		Structured	Concept-Level	Self-	Hierarchy-Aware
[L136]	System	Retrieval?	Extraction?	Evolving?	Context Selection?
[L137]					
[L138]	Traditional RAG	x	x	x	x
[L139]	RAPTOR	✓ (summaries)	x	x	x
[L140]	GraphRAG	✓ (subgraph)	✓ (partial)	x	x
[L141]	TreeRAG	✓ (tree)	x	x	x
[L142]	HiChunk	✓ (chunks)	x	x	x
[L143]					
[L144]	SHIA-RAG (Ours)	✓ (forest)	✓ (full)	✓	✓

[L147] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L149] ### 2.3 Literature Search Methodology

[L151] | Parameter | Details |

[L152] |-----|-----|

[L153] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L154] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L155] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L156] | **Time Range** | 2020–2026 |

[L157] | **Key Papers Reviewed** | RAPTOR (Sarthi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L161] ## 3. System Architecture

[L163] ### 3.1 Overview

[L165] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L167] | Phase | Layers | Mode | Purpose |

[L168] |-----|-----|-----|-----|

[L169] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L170] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L171] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L173] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L175]

```

[L176] graph TD
[L177]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L178]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Preprocessing<br/>Tokenization, normalization"]
[L179]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L180]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L181]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L182]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L182]     end

[L185]     subgraph "Phase 2: Online – Retrieval & Generation"
[L186]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, LSH"]
[L187]         KF --> L6
[L188]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L189]         L7 --> A["Answer + Citations"]
[L189]     end

[L192]     subgraph "Phase 3: Continuous – Operations"
[L193]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L194]         A --> L8
[L195]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L195]     end

```

[L199] ### 3.2 Data Flow

```

[L202] Raw Document (PDF/DOCX/HTML)
[L203]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L204] Normalized Text + Metadata
[L205]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L206] [Page → Region → Block] hierarchy
[L207]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L208] KnowledgeUnit objects (candidate concepts)
[L209]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L210] KnowledgeNode objects (canonical, deduplicated)
[L211]     ↓ Layer 4: Validate relations against ontology, score confidence
[L212] KnowledgeEdge objects (typed, scored)
[L213]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L214] Knowledge Forest (rooted trees + cross-links)
[L215]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L216] Selected context nodes (within token budget)
[L217]     ↓ Layer 7: Build prompt, invoke LLM
[L218]     ↓ Layer 7: Verify claims against evidence, bind citations
[L219] Answer with citations + confidence score
[L220]     ↓ Layer 8: Log, collect feedback, update forest weights

```

[L223] ### 3.3 Layer Summary

[L225] | Layer | Name | Input | Output | Key Algorithms |

[L226] |-----|-----|-----|-----|-----|

[L227] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L228] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L229] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L230] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L231] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L232] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L233] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L234] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L235] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L239] ## 4. Data Model — Layer 0

[L241] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` (concepts) and `KnowledgeEdge` (relationships).

[L243] ### 4.1 KnowledgeNode Schema

[L245] Each concept in the Knowledge Forest is represented as a `KnowledgeNode` :

[L247] ```json
{
[L249] "node_id": "NODE_000789",
[L250] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L251] "canonical_name": "Round Robin",
[L252] "aliases": ["RR", "Round-Robin Scheduling"],
[L253] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L254] "summaries": ["CPU scheduling method using time slices"],
[L255] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L256] "domain": "Operating Systems",
[L257] "properties": {
[L258] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L259] "preemptive": {"type": "boolean", "value": true}
},
[L261] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L262] "parent": "NODE_000456",
[L263] "children": ["NODE_001001", "NODE_001002"],
[L264] "related": ["NODE_000890"],
[L265] "incoming_edges": ["EDGE_001"],
[L266] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L267] "evidence": [

CHATGPT 19/08/2026, 22:45:38

{"paths":["files"]}

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file0

[L1] metadata:

[L2] query_result_index: 0

[L3] file_id: file_0000000033088207b45d2ae58cbd406c

[L4] version_id: 1

[L5] name: SHIA_RAG_Complete_Report(8).md
[L6] mime_type: text/markdown
[L7] surface: conversation
[L8] score: 0.03200204813108039
[L9] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--0
[L10] snippet:
[L11] # SHIA-RAG: Complete Project Report
[L12] ## Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation
[L13]
[L14] > **Project Type:** Minor Project (M.Tech)
[L15] > **Date:** August 2026
[L16] > **Documents Analyzed:** 6 PDFs + 914-page audit (total ~2M+ characters)
[L17] > **Audit Findings Fixed:** 75 issues across all 9 layers
[L18]
[L19] ---
[L20]
[L21] ## Table of Contents
[L22]
[L23] 1. Project Overview
[L24] 2. Document Inventory & Summary
[L25] 3. Critical Errors & Fixes
[L26] 4. Cross-Document Inconsistencies
[L27] 5. Novelty Contributions
[L28] - 5.1 Literature Search Methodology
[L29] - 5.2 Novelty Level Assessment
[L30] 6. Complete System Architecture
[L31] 7. Detailed Module Design
[L32] - 7.1 Layer 0: Core Data Model
[L33] - 7.2 Layer 1: Input Processing
[L34] - 7.3 Layer 2: Document Parsing
[L35] - 7.4 Layer 3: Knowledge Extraction
[L36] - 7.5 Layer 4: Relation Management
[L37] - 7.6 Layer 5: SHIA Core
[L38] - 7.7 Layer 6: Retrieval Engine
[L39] - 7.8 Layer 7: Generation Engine
[L40] - 7.9 Layer 8: Operations & Self-Evolution
[L41] - 7.10 Schema Validation
[L42] - 7.11 Database Indexing
[L43] - 7.12 Migration Strategy
[L44] - 7.13 Error Handling
[L45] - 7.14 Caching Strategy
[L46] - 7.15 Monitoring & Alerting
[L47] - 7.16 Multi-Tenancy
[L48] - 7.17 Async Processing

[L49] 8. Mathematical Formulations (Corrected)

[L50] 9. Implementation Roadmap

[L51] 10. Evaluation Framework

[L52] - 10.4 Statistical Significance

[L53] - 10.5 Error Analysis

[L54] - 10.6 Computational Cost

[L55] 11. Risk Analysis & Mitigation

[L56] 12. Conclusion

[L57]

[L58] ---

[L59]

[L60] ## 1. Project Overview

[L61]

[L62] ### 1.1 What is SHIA-RAG?

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite[turn1file1]`

[L63] metadata:

[L64] query_result_index: 1

[L65] file_id: file_00000000f620821186e54e582d59731e

[L66] version_id: 1

[L67] name: SHIA_RAG_Final_Report(3).md

[L68] mime_type: text/markdown

[L69] surface: conversation

[L70] score: 0.031544957774465976

[L71] document_chunk_id: file_00000000f620821186e54e582d59731e--1--0

[L72] snippet:

[L73] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L74]

[L75] > **Project Type:** Minor Project (M.Tech)

[L76] > **Date:** August 2026

[L77] > **Author:** [Your Name]

[L78]

[L79] ---

[L80]

[L81] ## Abstract

[L82]

[L83] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L84]

[L85] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a

collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L86]

[L87] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), the first benchmark specifically designed for hierarchical RAG quality.

[L88]

[L89] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L90]

[L91] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L92]

[L93] ---

[L94]

[L95] ## Table of Contents

[L96]

[L97] 1. [Introduction](#)

[L98] 2. [Literature Review](#)

[L99] 3. [System Architecture](#)

[L100] 4. [Data Model — Layer 0](#)

[L101] 5. [Offline Pipeline — Layers 1-5](#)

[L102] 6. [Online Pipeline — Layers 6-7](#)

[L103] 7. [Operations Layer — Layer 8](#)

[L104] 8. [Novel Contributions](#)

[L105] 9. [Mathematical Framework](#)

[L106] 10. [Implementation](#)

[L107] 11. [Evaluation Framework](#)

[L108] 12. [Risk Analysis](#)

[L109] 13. [Conclusion & Future Work](#)

[L110]

[L111] ---

[L112]

[L113] ## 1. Introduction

[L114]

[L115] ### 1.1 Problem Statement

[L116]

[L117] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L118]

[L119] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L120] 2. Embed each chunk into a vector space

[L121] 3. At query time, retrieve the top-k most similar chunks

[L122] 4. Feed these chunks as context to an LLM for answer generation

[L123]

[L124] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite[turn1file2]`

[L125] metadata:

[L126] query_result_index: 2

[L127] file_id: file_0000000033088207b45d2ae58cbd406c

[L128] version_id: 1

[L129] name: SHIA_RAG_Complete_Report(8).md

[L130] mime_type: text/markdown

[L131] surface: conversation

[L132] score: 0.031099324975891997

[L133] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--85

[L134] snippet:

[L135] ### 12.3 Next Steps

[L136]

[L137] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L138] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L139] 3. **Build Layers 1–2** (Document ingestion and parsing pipeline)

[L140] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L141] 5. **Build Layer 4** (Relation management with ontology validation)

[L142] 6. **Build Layer 5** (SHIA Core with corrected algorithms)

[L143] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L144] 8. **Build Layer 7** (Generation with verification)

[L145] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L146] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L147] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L148] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L149]

[L150] ### 12.4 Future Work

[L151]

[L152] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)

[L153] - **Multi-Tenancy**: Implement access control and document-level permissions

[L154] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets

[L155] - **Cross-Lingual Support**: Extend to multilingual documents and concept hierarchies

[L156] - **Incremental Updates**: Support adding new documents without full forest reconstruction

[L157] - **Advanced Verification**: Replace semantic similarity with NLI (Natural Language Inference) model for

claim verification

[L158] - **Exploration in Self-Evolution:** Add ϵ -greedy or Thompson sampling to edge weight updates to avoid exploitation-only bias

[L159]

[L160] ---

[L161]

[L162] > [!TIP]

[L163] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0–8) are now fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file3

[L164] metadata:

[L165] query_result_index: 3

[L166] file_id: file_0000000033088207b45d2ae58cbd406c

[L167] version_id: 1

[L168] name: SHIA_RAG_Complete_Report(8).md

[L169] mime_type: text/markdown

[L170] surface: conversation

[L171] score: 0.031024531024531024

[L172] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--2

[L173] snippet:

[L174] ### 1.1 What is SHIA-RAG?

[L175]

[L176] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) is a next-generation RAG system that fundamentally changes how information is retrieved for LLM-based question answering.

[L177]

[L178] **Traditional RAG** splits documents into flat text chunks → embeds them → retrieves top-k similar chunks → feeds to LLM.

[L179]

[L180] **SHIA-RAG** instead:

[L181] 1. **Extracts knowledge** (concepts, definitions, relations) from documents

[L182] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes (not text chunks)

[L183] 3. **Retrieves knowledge nodes** by traversing the hierarchy (parents, children, siblings)

[L184] 4. **Assembles context** intelligently within a token budget

[L185] 5. **Generates answers** with citation-backed evidence

[L186]

[L187] ### 1.2 Why It Matters

[L188]

[L189] | Problem with Traditional RAG | How SHIA-RAG Solves It |

[L190] |---|---|

[L191] | Chunks break semantic continuity | Knowledge nodes preserve complete concepts |

[L192] | Duplicate/fragmented information | Entity resolution merges synonyms into single nodes |

[L193] | Fixed chunk granularity | Hierarchy allows multi-level retrieval (broad → specific) |

[L194] | Weak multi-hop reasoning | Parent-child paths encode reasoning chains |

[L195] | No cross-document fusion | Knowledge Forest merges concepts across all documents |

[L196]

[L197] ### 1.3 Core Innovation

[L198]

[L199] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "CS → OS → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L200]

[L201] ---

[L202]

[L203] ## 2. Document Inventory & Summary

[L204]

[L205] ### 2.1 All 6 PDFs Analyzed

[L206]

[L207] | # | File Name | Pages | Content Type | What It Contains |

[L208] |---|-----|-----|-----|-----|

[L209] | 1 | first Pdf.pdf | 24 | Original MDS (Academic) | Executive Summary, Problem Statement, Literature Review, System Architecture (13 modules), SHIA Core algorithms (CPG++, PRE, HV, FI, CLD), Evaluation Plan, Implementation Roadmap |

[L210] | 2 | third Pdf.pdf | 10 | Algorithms & Scoring | Detailed scoring formulas for all SHIA modules, pseudocode for 6 algorithms, retrieval policies (RL-based adaptive depth), uncertainty resolution, security considerations |

[L211] | 3 | 4h pdf.pdf | 22 | Formal Specification | Formal mathematical definitions, Energy Minimization objective, C++ interfaces, SHEF evaluation framework, complexity analysis, experimental methodology, reproducibility checklist |

[L212] | 4 | Master Design Specification (SHIA-RAG).pdf | 46 | Production MDS | Complete 8-layer architecture with JSON schemas, SQL/Cypher DB design, OpenAPI contracts, microservice folder structure, Layer-by-layer algorithms with pseudocode |

[L213] | 5 | PDF Comparison Summary.pdf | 236 | Research Comparison | Deep comparison of all PDFs, literature positioning against TreeRAG/HiChunk/RAPTOR/GraphRAG, gap analysis, combined architecture walkthrough |

[L214] | 6 | PDFs discussion and analysis.pdf | 483 | Extended Analysis | Collaborative transcript refining SHIA-RAG, detailed layer-by-layer walkthroughs, critical review identifying gaps, research contribution proposals |

[L215]

[L216] ### 2.2 How Documents Relate

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite[turn1file4]`

[L217] metadata:

[L218] query_result_index: 4

[L219] file_id: file_0000000033088207b45d2ae58cbd406c

[L220] version_id: 1

[L221] name: SHIA_RAG_Complete_Report(8).md
[L222] mime_type: text/markdown
[L223] surface: conversation
[L224] score: 0.030798389007344232
[L225] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--82
[L226] snippet:
[L227]

[L228] Expected Cost Table:

[L229]

[L230] Operation	[L231] SHIA-RAG	[L232] Flat RAG (base)
[L233] Index 1K documents	[L234] ~\$X.XX	[L235] ~\$Y.YY
[L236] 1K queries	[L237] ~\$X.XX	[L238] ~\$Y.YY
[L239] GPU hours (indexing)	[L240] ~X.X hrs	[L241] ~Y.Y hrs
[L242] Peak memory	[L243] ~X.X GB	[L244] ~Y.Y GB

[L238]
[L239] ---
[L240]
[L241] ## 11. Risk Analysis & Mitigation
[L242]
[L243] | Risk | Impact | Probability | Mitigation |
[L244] |-----|-----|-----|-----|
[L245] | LLM extraction errors propagate to forest | High | High | Confidence thresholds (KCE), human-in-loop review for low-confidence nodes |
[L246] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |
[L247] | O(N²) operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |
[L248] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning |
[L249] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |
[L250] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |
[L251] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |
[L252]
[L253] ---
[L254]
[L255] ## 12. Conclusion
[L256]
[L257] ### 12.1 Summary of Work Done
[L258]
[L259] 1.  **Read and analyzed all 6 source PDFs + 914-page audit** (~2M+ characters total)

- [L260] 2. **Identified and fixed 75 issues** across all 9 layers (Layer 0–8), including 38 critical bugs, 25 important gaps, 7 ambiguities
- [L261] 3. **Fixed all formula contradictions:** PRE depth formula unified to $(D_{\max} - \text{depth}) / D_{\max}$, self-evolution feedback formula unified to $\exp(\eta \cdot f)$
- [L262] 4. **Resolved cross-document inconsistencies** (3 PRE formulas → 1, 13 modules → 8 layers, single-parent constraint declared)
- [L263] 5. **Added 5 novel contributions** with literature search methodology and novelty level assessment
- [L264] 6. **Designed complete implementation architecture** with folder structure, tech stack, .gitignore, CI/CD
- [L265] 7. **Created evaluation framework** (SHEF) with metrics, datasets, baselines, statistical significance testing, error analysis, and computational cost analysis
- [L266] 8. **Planned 6-month implementation roadmap** with clear milestones
- [L267] 9. **Added detailed sections for ALL layers** (Layers 1–4 were previously missing dedicated sections)
- [L268] 10. **Added Layer 7 complete implementation:** prompt template, claim extraction, evidence verification, citation binding, confidence formula, unsupported/partial/empty-context policies, structured output contract
- [L269] 11. **Added Layer 8 complete implementation:** logging schema, feedback acquisition, feedback attribution isolation, self-evolution loop closure (edge weights → utility() → Tree Knapsack), monitoring metrics, weight normalization/decay
- [L270] 12.

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite{turn1file5}`

[L271] metadata:

[L272] query_result_index: 5

[L273] file_id: file_0000000033088207b45d2ae58cbd406c

[L274] version_id: 1

[L275] name: SHIA_RAG_Complete_Report(8).md

[L276] mime_type: text/markdown

[L277] surface: conversation

[L278] score: 0.030621785881252923

[L279] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--83

[L280] snippet:

[L281] idempotent operations |

[L282]

[L283] ---

[L284]

[L285] ## 12. Conclusion

[L286]

[L287] ### 12.1 Summary of Work Done

[L288]

[L289] 1. **Read and analyzed all 6 source PDFs + 914-page audit** (~2M+ characters total)

[L290] 2. **Identified and fixed 75 issues** across all 9 layers (Layer 0–8), including 38 critical bugs, 25 important gaps, 7 ambiguities

- [L291] 3. **Fixed all formula contradictions:** PRE depth formula unified to $(D_{\max} - \text{depth}) / D_{\max}$, self-evolution feedback formula unified to $\exp(\eta \cdot f)$
- [L292] 4. **Resolved cross-document inconsistencies** (3 PRE formulas $\rightarrow$ 1, 13 modules $\rightarrow$ 8 layers, single-parent constraint declared)
- [L293] 5. **Added 5 novel contributions** with literature search methodology and novelty level assessment
- [L294] 6. **Designed complete implementation architecture** with folder structure, tech stack, .gitignore, CI/CD
- [L295] 7. **Created evaluation framework** (SHEF) with metrics, datasets, baselines, statistical significance testing, error analysis, and computational cost analysis
- [L296] 8. **Planned 6-month implementation roadmap** with clear milestones
- [L297] 9. **Added detailed sections for ALL layers** (Layers 1–4 were previously missing dedicated sections)
- [L298] 10. **Added Layer 7 complete implementation:** prompt template, claim extraction, evidence verification, citation binding, confidence formula, unsupported/partial/empty-context policies, structured output contract
- [L299] 11. **Added Layer 8 complete implementation:** logging schema, feedback acquisition, feedback attribution isolation, self-evolution loop closure (edge weights $\rightarrow$ utility() $\rightarrow$ Tree Knapsack), monitoring metrics, weight normalization/decay
- [L300] 12. **Defined all previously undefined functions:** `flag_for_restructure()`, `rebalance_forest()`, `verify_claim()`, `compute_answer_confidence()`, `format_citation()`, `assemble_context()`, `multi_tree_knapsack()`, `layer1_process()`, `layer2_parse()`, `layer3_extract()`, `layer4_relate()`, `alias_resolution()`
- [L301] 13. **Added missing schemas:** `KnowledgeUnit`, `NormalizedDocument`, `DocumentBlock`, `RetrievalResult`, `GenerationResult`, `VerifiedClaim`, `QueryEvent`, `QueryLog`
- [L302] 14. **Defined complete relation ontology:** 12 relation types across hierarchical (IS_A, PART_OF, SUBCLASS_OF, INSTANCE_OF) and semantic (USES, CAUSES, DEPENDS_ON, CONTRASTS_WITH, SIMILAR_TO, HAS_PROPERTY, PRODUCES, PREREQUISITE) categories
- [L303]
- [L304] ### 12.2 Key Novelties (Paper-Worthy)
- [L305]
- [L306] | # | Novelty | Level | Why It's New |
- [L307] |---|-----|-----|-----|
- [L308] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |
- [L309] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |
- [L310] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |
- [L311] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |
- [L312] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |
- [L313]
- [L314] ### 12.3 Next Steps

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file6

[L315] metadata:

[L316] query_result_index: 6

[L317] file_id: file_00000000f620821186e54e582d59731e

[L318] version_id: 1

[L319] name: SHIA_RAG_Final_Report(3).md

[L320] mime_type: text/markdown

[L321] surface: conversation

[L322] score: 0.030090497737556562

[L323] document_chunk_id: file_00000000f620821186e54e582d59731e--1--39

[L324] snippet:

[L325] | Metric | How Measured | Report Format |

[L326] |-----|-----|-----|

[L327] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |

[L328] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |

[L329] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |

[L330] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |

[L331] | Memory usage | Peak RAM + VRAM | GB |

[L332] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |

[L333]

[L334] ---

[L335]

[L336] ## 12. Risk Analysis

[L337]

[L338] | Risk | Impact | Probability | Mitigation |

[L339] |-----|-----|-----|

[L340] | LLM extraction errors propagate to forest | High | High | Confidence thresholds, human-in-loop review for low-confidence nodes |

[L341] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L342] | O(N²) operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L343] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning |

[L344] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L345] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L346] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L347]

[L348] ---

[L349]

[L350] ## 13. Conclusion & Future Work

[L351]

[L352] ### 13.1 Summary

[L353]

[L354] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L355]

[L356] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem

[L357] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L358] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction

[L359] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation

[L360] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality

[L361]

[L362] The complete system specification covers:

[L363] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L364] - **12 defined functions** with complete implementations

[L365] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L366] - **8 schemas** covering all data objects from document blocks to query logs

[L367] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L368] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L369]

[L370] ### 13.2 Key Novelties

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite[turn1file7]`

[L371] metadata:

[L372] query_result_index: 7

[L373] file_id: file_00000000f620821186e54e582d59731e

[L374] version_id: 1

[L375] name: SHIA_RAG_Final_Report(3).md

[L376] mime_type: text/markdown

[L377] surface: conversation

[L378] score: 0.02964426877470356

[L379] document_chunk_id: file_00000000f620821186e54e582d59731e--1--2

[L380] snippet:

[L381] | Problem | Description | Impact |

[L382] |-----|-----|-----|

[L383] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L384] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight

variations | Token budget wasted on redundancy |

[L385] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L386] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |

[L387] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L388]

[L389] ### 1.2 Proposed Solution: SHIA-RAG

[L390]

[L391] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L392]

[L393]

[L394] Traditional RAG: Document $\rightarrow$ [Chunk₁, Chunk₂, ..., Chunk_n] $\rightarrow$ Top-k $\rightarrow$ LLM
[L395] SHIA-RAG: Document $\rightarrow$ Knowledge Forest (concepts + relations) $\rightarrow$ Adaptive Traversal $\rightarrow$ LLM
[L396]

[L397]

[L398] The system:

[L399] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L400] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L401] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L402] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L403] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L404] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L405]

[L406] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science $\rightarrow$ Operating Systems $\rightarrow$ Scheduling $\rightarrow$ Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L407]

[L408] ### 1.3 Objectives

[L409]

[L410] 1. Design a complete 8-layer architecture for hierarchical RAG

[L411] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L412] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L413] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L414] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L415] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L416]

[L417] ### 1.4 Contributions

[L418]

[L419] This work makes five novel contributions to the RAG literature:

[L420]

[L421] | # | Contribution | Novelty Level |

[L422] |---|-----|-----|

[L423] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L424] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L425] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L426] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L427] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L428]

[L429] ---

[L430]

[L431] ## 2. Literature Review

[L432]

[L433] ### 2.1 Related Work

[L434]

[L435] #### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite[turn1file8]`

[L436] metadata:

[L437] query_result_index: 8

[L438] file_id: file_00000000f620821186e54e582d59731e

[L439] version_id: 1

[L440] name: SHIA_RAG_Final_Report(3).md

[L441] mime_type: text/markdown

[L442] surface: conversation

[L443] score: 0.029386529386529386

[L444] document_chunk_id: file_00000000f620821186e54e582d59731e--1--40

[L445] snippet:

[L446] temporal versioning |

[L447] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L448] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L449] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L450]

[L451] ---

[L452]

[L453] ## 13. Conclusion & Future Work

[L454]

[L455] ### 13.1 Summary

[L456]

[L457] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L458]

[L459] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem

[L460] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L461] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction

[L462] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation

[L463] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality

[L464]

[L465] The complete system specification covers:

[L466] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L467] - **12 defined functions** with complete implementations

[L468] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L469] - **8 schemas** covering all data objects from document blocks to query logs

[L470] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L471] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L472]

[L473] ### 13.2 Key Novelties

[L474]

[L475] | # | Novelty | Level | Why It's New |

[L476] |---|-----|-----|-----|

[L477] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L478] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L479] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L480] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L481] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L482]

[L483] ### 13.3 Next Steps

[L484]

[L485] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L486] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

- [L487] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)
- [L488] 4. **Build Layer 3** (Knowledge extraction with LLM)
- [L489] 5. **Build Layer 4** (Relation management with ontology validation)
- [L490] 6. **Build Layer 5** (SHIA Core with algorithms)
- [L491] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
- [L492] 8. **Build Layer 7** (Generation with verification)
- [L493] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
- [L494] 10. **Run first evaluation** on OS textbook dataset using SHEF
- [L495] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
- [L496] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
- [L497]
- [L498] ### 13.4 Future Work

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file9

- [L499] metadata:
- [L500] query_result_index: 9
- [L501] file_id: file_00000000f620821186e54e582d59731e
- [L502] version_id: 1
- [L503] name: SHIA_RAG_Final_Report(3).md
- [L504] mime_type: text/markdown
- [L505] surface: conversation
- [L506] score: 0.029206349206349208
- [L507] document_chunk_id: file_00000000f620821186e54e582d59731e--1--3
- [L508] snippet:
- [L509] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.
- [L510]
- [L511] ### 1.3 Objectives
- [L512]
- [L513] 1. Design a complete 8-layer architecture for hierarchical RAG
- [L514] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)
- [L515] 3. Implement a self-evolving feedback loop that improves retrieval quality over time
- [L516] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)
- [L517] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality
- [L518] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans
- [L519]
- [L520] ### 1.4 Contributions
- [L521]
- [L522] This work makes five novel contributions to the RAG literature:
- [L523]
- [L524] | # | Contribution | Novelty Level |
- [L525] |---|-----|-----|

[L526] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L527] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L528] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L529] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L530] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L531]

[L532] ---

[L533]

[L534] ## 2. Literature Review

[L535]

[L536] ### 2.1 Related Work

[L537]

[L538] ##### 2.1.1 Flat-Chunk RAG Systems

[L539]

[L540] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Guu et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-reflection, but all operate on flat text chunks with no structural awareness.

[L541]

[L542] ##### 2.1.2 Structured RAG Systems

[L543]

[L544] Recent work has begun exploring structured retrieval:

[L545]

[L546] | System | Approach | Key Limitation |

[L547] |-----|-----|-----|

[L548] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L549] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L550] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L551] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L552]

[L553] ##### 2.1.3 Knowledge Graph Construction

[L554]

[L555] Ontology learning and taxonomy induction have a rich history in NLP:

[L556] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L557] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L558] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L559]

[L560] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-**

evolution.

[L561]

[L562] ### 2.2 Research Gap

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite{turn1file10}`

[L563] metadata:

[L564] query_result_index: 10

[L565] file_id: file_0000000033088207b45d2ae58cbd406c

[L566] version_id: 1

[L567] name: SHIA_RAG_Complete_Report(8).md

[L568] mime_type: text/markdown

[L569] surface: conversation

[L570] score: 0.027984344422700584

[L571] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--3

[L572] snippet:

[L573] ---

[L574]

[L575] ## 2. Document Inventory & Summary

[L576]

[L577] ### 2.1 All 6 PDFs Analyzed

[L578]

[L579] | # | File Name | Pages | Content Type | What It Contains |

[L580] |---|-----|-----|-----|-----|

[L581] | 1 | first Pdf.pdf | 24 | Original MDS (Academic) | Executive Summary, Problem Statement, Literature Review, System Architecture (13 modules), SHIA Core algorithms (CPG++, PRE, HV, FI, CLD), Evaluation Plan, Implementation Roadmap |

[L582] | 2 | third Pdf.pdf | 10 | Algorithms & Scoring | Detailed scoring formulas for all SHIA modules, pseudocode for 6 algorithms, retrieval policies (RL-based adaptive depth), uncertainty resolution, security considerations |

[L583] | 3 | 4h pdf.pdf | 22 | Formal Specification | Formal mathematical definitions, Energy Minimization objective, C++ interfaces, SHEF evaluation framework, complexity analysis, experimental methodology, reproducibility checklist |

[L584] | 4 | Master Design Specification (SHIA-RAG).pdf | 46 | Production MDS | Complete 8-layer architecture with JSON schemas, SQL/Cypher DB design, OpenAPI contracts, microservice folder structure, Layer-by-layer algorithms with pseudocode |

[L585] | 5 | PDF Comparison Summary.pdf | 236 | Research Comparison | Deep comparison of all PDFs, literature positioning against TreeRAG/HiChunk/RAPTOR/GraphRAG, gap analysis, combined architecture walkthrough |

[L586] | 6 | PDFs discussion and analysis.pdf | 483 | Extended Analysis | Collaborative transcript refining SHIA-RAG, detailed layer-by-layer walkthroughs, critical review identifying gaps, research contribution proposals |

[L587]

[L588] ### 2.2 How Documents Relate

[L589]

[L590]

```
[L591] graph TD
[L592]   A["PDF 1: first Pdf.pdf<br/>(24p) Original Vision"] --> D["PDF 5: Comparison Summary<br/>(23
[L593]   B["PDF 2: third Pdf.pdf<br/>(10p) Algorithms"] --> D
[L594]   C["PDF 3: 4h pdf.pdf<br/>(22p) Formal Spec"] --> D
[L595]   E["PDF 4: Master Design Spec<br/>(46p) Production Blueprint"] --> D
[L596]   D --> F["PDF 6: Discussion & Analysis<br/>(483p) Critical Refinement"]
[L597]
[L598]   A -->|"Evolved into"| E
[L599]   B -->|"Adds formulas to"| E
[L600]   C -->|"Adds rigor to"| E
[L601]   F -->|"Identifies gaps in"| A
[L602]   F -->|"Identifies gaps in"| E
[L603]
```

[L604]

[L605] ---

[L606]

[L607] ## 3. Critical Errors & Fixes

[L608]

[L609] > [!CAUTION]

[L610] > **24+ issues identified across all 6 PDFs.** Below are the most critical ones with proposed fixes.

[L611]

[L612] ### 3.1 FATAL: Broken Cycle Detection (PDF 2 — third Pdf.pdf)

[L613]

[L614] **The Bug:** The DFS cycle detection algorithm traverses the **wrong direction**.

[L615]

[L616] ```diff

[L617] # BROKEN (current code)

[L618] - def is_valid(N, P, forest):

[L619] - stack = [P] # Start from Parent

[L620] - while stack:

[L621] - cur = stack.pop()

[L622] - for child in forest.children(cur): # Go DOWN from P

[L623] - if child == N: # Found N as descendant of P

[L624] - return False # This detects redundancy, NOT cycles

[L625] - stack.append(child)

[L626] - return True

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file11

[L627] metadata:

[L628] query_result_index: 11

[L629] file_id: file_0000000033088207b45d2ae58cbd406c

[L630] version_id: 1

[L631] name: SHIA_RAG_Complete_Report(8).md

[L632] mime_type: text/markdown

[L633] surface: conversation

[L634] score: 0.027623285131627734

[L635] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--19

[L636] snippet:

[L637] ---

[L638]

[L639] **### Novelty 3: Multi-Objective Hierarchy Energy Minimization**

[L640]

[L641] **What exists:** TreeRAG and RAPTOR use simple clustering. GraphRAG uses flat triple extraction.

[L642]

[L643] **What SHIA-RAG proposes:** Hierarchy construction as a formal **energy minimization** problem:

[L644]

[L645]
$$J(\mathcal{F}) = \sum_{(P_i, N_i)} \left[\lambda_1 \cdot L_{\text{sem}}(P_i, N_i) + \lambda_2 \cdot L_{\text{depth}}(N_i) + \lambda_3 \cdot L_{\text{conf}}(P_i, N_i) + \lambda_4 \cdot L_{\text{onto}}(P_i, N_i) \right]$$

[L646]

[L647] Where:

[L648] - L_{sem} = semantic distance loss (1 - cosine similarity)

[L649] - L_{depth} = depth regularization (penalizes too-deep or too-shallow trees)

[L650] - L_{conf} = confidence loss (low-confidence placements increase energy)

[L651] - L_{onto} = ontological consistency loss (parent should be more general)

[L652]

[L653] **Why novel:** No RAG paper has formalized hierarchy construction as a multi-objective optimization problem for hierarchy induction in RAG. The formulation enables principled trade-offs between semantic fit, structural balance, confidence, and ontological consistency.

[L654]

[L655] > [!NOTE]

[L656] > The L_{onto} term uses a **sigmoid relaxation** $\sigma(\text{type_score}(P) - \text{type_score}(N))$ instead of a non-differentiable indicator function. See Section 8.1 for the corrected formulation.

[L657]

[L658] ---

[L659]

[L660] **### Novelty 4: Adaptive Depth Traversal via Query Complexity Classification**

[L661]

[L662] **What exists:** All RAG systems retrieve a fixed number of chunks/nodes regardless of query complexity.

[L663]

[L664] **What SHIA-RAG proposes:** A **query complexity classifier** that dynamically adjusts traversal depth:

[L665]

[L666] | Query Type | Example | Traversal Strategy |

[L667] |-----|-----|-----|

[L668] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |

[L669] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |

[L670] | MULTI-HOP | "How does OSPF convergence affect network reliability?" | Deep: Full path traversal + cross-links (depth=3+) |

[L671] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Retrieve at root/topic level, expand selectively |

[L672]

[L673] ```python

[L674] class QueryComplexityClassifier:

[L675] def classify(self, query):

[L676] features = {

[L677] 'num_entities': count_entities(query),

[L678] 'has_comparison': any(w in query.lower() for w in ['compare', 'vs', 'difference']),

[L679] 'has_causal': any(w in query.lower() for w in ['why', 'how', 'cause', 'effect']),

[L680] 'num_hops_estimated': estimate_reasoning_hops(query),

[L681] }

[L682] # Lightweight classifier (fine-tuned BERT or rule-based)

[L683] return self.model.predict(features) # → SIMPLE | MODERATE | COMPLEX | SYNTHESIS

[L684]

[L685] def estimate_reasoning_hops(query):

[L686] """Rule-based heuristic to estimate the number of reasoning hops

[L687] required to answer a query. Returns an integer in [1, 5]."""

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file12

[L688] metadata:

[L689] query_result_index: 12

[L690] file_id: file_00000000f620821186e54e582d59731e

[L691] version_id: 1

[L692] name: SHIA_RAG_Final_Report(3).md

[L693] mime_type: text/markdown

[L694] surface: conversation

[L695] score: 0.027529761904761904

[L696] document_chunk_id: file_00000000f620821186e54e582d59731e--1--38

[L697] snippet:

[L698] 1. **Retrieval Quality:** Compare Recall@k, NDCG@k, Hop Accuracy across all baselines

[L699] 2. **Answer Quality:** Compare F1, BLEU, BERTScore for generated answers

[L700] 3. **Hierarchy Quality:** Parent Assignment Accuracy, Tree Edit Distance vs gold taxonomy

[L701] 4. **Ablation Studies:**

[L702] - Remove CPG++ → use random parent selection

[L703] - Remove adaptive depth → use fixed depth=2

[L704] - Remove Tree Knapsack → use flat top-k

[L705] - Remove self-evolution → static forest

[L706] 5. **Scalability:** Measure indexing time, query latency, memory usage at 100/1K/10K/100K nodes

[L707]

[L708] ### 11.4 Statistical Significance Testing

[L709]

[L710] | Test | Application | Details |

[L711] |-----|-----|-----|

[L712] | Paired t-test | SHIA-RAG vs each baseline | Per-query metric comparison |

[L713] | Significance level | All tests | $\alpha = 0.05$ |

[L714] | Effect size | All comparisons | Cohen's d (small: 0.2, medium: 0.5, large: 0.8) |

[L715] | Confidence intervals | All reported metrics | 95% CI via bootstrap (1000 resamples) |

[L716] | Multiple comparison correction | When comparing 5+ baselines | Bonferroni or Holm-Bonferroni |

[L717]

[L718] ### 11.5 Error Analysis

[L719]

[L720] | Analysis Type | Method | Details |

[L721] |-----|-----|-----|

[L722] | Query categorization | Classify all test queries | DEFINITION, COMPARISON, MULTI-HOP, SYNTHESIS |

[L723] | Failure pattern analysis | Cluster failed queries | Identify common failure modes |

[L724] | Qualitative analysis | Manual inspection | Review 100 randomly sampled failed queries |

[L725] | Per-category breakdown | Stratified metrics | Report accuracy per query type |

[L726]

[L727] ### 11.6 Computational Cost Analysis

[L728]

[L729] | Metric | How Measured | Report Format |

[L730] |-----|-----|-----|

[L731] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |

[L732] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |

[L733] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |

[L734] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |

[L735] | Memory usage | Peak RAM + VRAM | GB |

[L736] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |

[L737]

[L738] ---

[L739]

[L740] ## 12. Risk Analysis

[L741]

[L742] | Risk | Impact | Probability | Mitigation |

[L743] |-----|-----|-----|-----|

[L744] | LLM extraction errors propagate to forest | High | High | Confidence thresholds, human-in-loop review for low-confidence nodes |

[L745] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L746] | $O(N^2)$ operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L747] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning |

[L748] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L749] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L750] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L751]

[L752] ---

[L753]

[L754] ## 13. Conclusion & Future Work

[L755]

[L756] ### 13.1 Summary

[L757]

[L758] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file13

[L759] metadata:

[L760] query_result_index: 13

[L761] file_id: file_0000000033088207b45d2ae58cbd406c

[L762] version_id: 1

[L763] name: SHIA_RAG_Complete_Report(8).md

[L764] mime_type: text/markdown

[L765] surface: conversation

[L766] score: 0.027425373134328357

[L767] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--81

[L768] snippet:

[L769] link between "TCP" and |

[L770] | | "Congestion Control" |

[L771]

|-----|

[L772] | Over-merging | | "Stack (data structure)" merged |

[L773] | (alias resolution) | | with "Stack (networking)" |

[L774]

|-----|

[L775] | Insufficient depth | | Query needed depth=3 but |

[L776] | (adaptive traversal) | | classifier chose depth=1 |

[L777]

|-----|

[L778]

```
[L779]
[L780] ### 10.6 Computational Cost Analysis
[L781]
[L782] | Metric | How Measured | Report Format |
[L783] |-----|-----|-----|
[L784] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |
[L785] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |
[L786] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |
[L787] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |
[L788] | Memory usage | Peak RAM + VRAM | GB |
[L789] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |
[L790]
[L791]
```

text

[L792] Expected Cost Table:

[L793]			
[L794]	Operation	SHIA-RAG	Flat RAG (base)
[L795]			
[L796]	Index 1K documents	~\$X.XX	~\$Y.YY
[L797]	1K queries	~\$X.XX	~\$Y.YY
[L798]	GPU hours (indexing)	~X.X hrs	~Y.Y hrs
[L799]	Peak memory	~X.X GB	~Y.Y GB
[L800]			

[L801] ```

[L802]

[L803] ---

[L804]

[L805] ## 11. Risk Analysis & Mitigation

[L806]

[L807] | Risk | Impact | Probability | Mitigation |

[L808] |-----|-----|-----|-----|

[L809] | LLM extraction errors propagate to forest | High | High | Confidence thresholds (KCE), human-in-loop review for low-confidence nodes |

[L810] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L811] | O(N²) operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L812] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning |

[L813] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L814] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L815] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L816]

[L817] ---

[L818]

[L819] ## 12. Conclusion

[L820]

[L821] ### 12.1 Summary of Work Done

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite[turn1file14]`

[L822] metadata:

[L823] query_result_index: 14

[L824] file_id: file_0000000033088207b45d2ae58cbd406c

[L825] version_id: 1

[L826] name: SHIA_RAG_Complete_Report(8).md

[L827] mime_type: text/markdown

[L828] surface: conversation

[L829] score: 0.02690501986276634

[L830] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--1

[L831] snippet:

[L832] 6 Layer 5: SHIA Core](#76-layer-5-shia-core-corrected-pipeline)

[L833] - 7.7 Layer 6: Retrieval Engine

[L834] - 7.8 Layer 7: Generation Engine

[L835] - 7.9 Layer 8: Operations & Self-Evolution

[L836] - 7.10 Schema Validation

[L837] - 7.11 Database Indexing

[L838] - 7.12 Migration Strategy

[L839] - 7.13 Error Handling

[L840] - 7.14 Caching Strategy

[L841] - 7.15 Monitoring & Alerting

[L842] - 7.16 Multi-Tenancy

[L843] - 7.17 Async Processing

[L844] 8. Mathematical Formulations (Corrected)

[L845] 9. Implementation Roadmap

[L846] 10. Evaluation Framework

[L847] - 10.4 Statistical Significance

[L848] - 10.5 Error Analysis

[L849] - 10.6 Computational Cost

[L850] 11. Risk Analysis & Mitigation

[L851] 12. Conclusion

[L852]

[L853] ---

[L854]

[L855] ## 1. Project Overview

[L856]

[L857] ### 1.1 What is SHIA-RAG?

[L858]

[L859] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) is a next-

generation RAG system that fundamentally changes how information is retrieved for LLM-based question answering.

[L860]

[L861] **Traditional RAG** splits documents into flat text chunks → embeds them → retrieves top-k similar chunks → feeds to LLM.

[L862]

[L863] **SHIA-RAG** instead:

[L864] 1. **Extracts knowledge** (concepts, definitions, relations) from documents

[L865] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes (not text chunks)

[L866] 3. **Retrieves knowledge nodes** by traversing the hierarchy (parents, children, siblings)

[L867] 4. **Assembles context** intelligently within a token budget

[L868] 5. **Generates answers** with citation-backed evidence

[L869]

[L870] ### 1.2 Why It Matters

[L871]

[L872] | Problem with Traditional RAG | How SHIA-RAG Solves It |

[L873] |---|---|

[L874] | Chunks break semantic continuity | Knowledge nodes preserve complete concepts |

[L875] | Duplicate/fragmented information | Entity resolution merges synonyms into single nodes |

[L876] | Fixed chunk granularity | Hierarchy allows multi-level retrieval (broad → specific) |

[L877] | Weak multi-hop reasoning | Parent-child paths encode reasoning chains |

[L878] | No cross-document fusion | Knowledge Forest merges concepts across all documents |

[L879]

[L880] ### 1.3 Core Innovation

[L881]

[L882] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "CS → OS → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L883]

[L884] ---

[L885]

[L886] ## 2. Document Inventory & Summary

[L887]

[L888] ### 2.1 All 6 PDFs Analyzed

[L889]

[L890] | # | File Name | Pages | Content Type | What It Contains |

[L891] |---|-----|-----|-----|-----|

[L892] | 1 | `first Pdf.

DEEP RESEARCH 19/08/2026, 22:45:44

Citation Marker: `filecite`turn1file15

[L893] metadata:

[L894] query_result_index: 15

[L895] file_id: file_00000000f620821186e54e582d59731e

[L896] version_id: 1

[L897] name: SHIA_RAG_Final_Report(3).md

[L898] mime_type: text/markdown

[L899] surface: conversation

[L900] score: 0.0266900790166813

[L901] document_chunk_id: file_00000000f620821186e54e582d59731e--1--30

[L902] snippet:

[L903] **SHIA-RAG approach:** A query complexity classifier that dynamically adjusts traversal depth:

[L904]

[L905] | Query Type | Example | Traversal Strategy |

[L906] |-----|-----|-----|

[L907] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |

[L908] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |

[L909] | MULTI-HOP | "How does OSPF convergence affect reliability?" | Deep: Full path traversal + cross-links (depth=3+) |

[L910] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Root-level retrieval, expand selectively |

[L911]

[L912] **Novelty level:** Incremental — novel in the specific application to hierarchical RAG traversal depth.

[L913]

[L914] ---

[L915]

[L916] ### 8.5 Novelty 5: SHEF — SHIA-RAG Evaluation Framework

[L917]

[L918] **Existing approach:** Standard RAG benchmarks (MS MARCO, NaturalQuestions) only measure flat retrieval. No benchmark tests hierarchical retrieval quality.

[L919]

[L920] **SHIA-RAG approach:** A novel evaluation framework with 3 metric categories:

[L921]

[L922] #### A. Retrieval Metrics

[L923] | Metric | Formula | What It Measures |

[L924] |-----|-----|-----|

[L925] | Hop Accuracy | correct multi-hop answers / total multi-hop queries | Multi-hop reasoning quality |

[L926] | Ancestor Recall | retrieved ancestors $\cap$ gold ancestors / gold ancestors | Correct concept chain retrieval |

[L927] | Context Coherence | Avg semantic similarity between consecutive context nodes | Logical coherence of assembled context |

[L928]

[L929] #### B. Hierarchy Metrics

[L930] | Metric | Formula | What It Measures |

[L931] |-----|-----|-----|

[L932] | Parent Assignment Accuracy | correctly placed nodes / total nodes | Quality of hierarchy induction |

[L933] | Tree Edit Distance | Normalized TED vs gold taxonomy | Structural similarity to human taxonomy |

[L934] | Depth Distribution Score | KL-divergence from ideal depth distribution | Tree balance |

[L935]

[L936] #### C. Forest Quality Metrics

[L937] | Metric | Formula | What It Measures |

[L938] |-----|-----|-----|

[L939] | Cross-Doc Fusion Accuracy | correctly merged nodes / total mergeable pairs | Entity resolution quality |

[L940] | Orphan Rate | root nodes without children / total root nodes | Isolated concept rate |

[L941] | Forest Density | edges / nodes | Knowledge forest connectivity |

[L942]

[L943] **Novelty level:** Incremental (Novel Combination) — first benchmark specifically designed for hierarchical RAG quality.

[L944]

[L945] ### 8.6 Novelty Level Assessment Summary

[L946]

[L947] | # | Novelty | Level | Justification |

[L948] |---|-----|-----|-----|

[L949] | 1 | Tree Knapsack for Context Selection | **Incremental (Novel Application)** | Adaptation of existing OR technique to a new domain (RAG context selection) |

[L950] | 2 | Self-Evolving Knowledge Forest | **Moderately Novel** | Novel combination of retrieval feedback + hierarchy restructuring |

[L951] | 3 | Multi-Objective Energy Minimization | **Moderately Novel** | Combined formulation for RAG hierarchy induction is new |

[L952] | 4 | Adaptive Depth Traversal | **Incremental** | Novel application to hierarchical RAG traversal depth |

[L953] | 5 | SHEF Evaluation Framework | **Incremental (Novel Combination)** | Standard metrics adapted as a unified framework for hierarchical RAG |

[L954]

[L955] ---

[L956]

[L957] ## 9. Mathematical Framework

[L958]

[L... (truncated)

CHATGPT 19/08/2026, 22:45:44

Haan. Maine dono files ko read karke unka overall structure aur detailed content analyze karna start kar diya hai. Ye sirf do similar reports nahi hain—dono ka role thoda different hai.

Dono files ka role

1. Complete Report

Ye **master / technical source of truth** jaisa hai. Isme project ko refine karte waqt:

- 6 source PDFs ka analysis
- cross-document inconsistencies
- critical bugs aur fixes
- corrected algorithms
- detailed Layer 0–8 design
- schemas
- formulas

- implementation contracts
- database, caching, monitoring, async processing
- mathematical corrections
- evaluation
- risks

sab include kiya gaya hai. Report ke according, multiple source documents aur audit ko reconcile karke issues/fixes aur final architecture consolidate ki gayi hai. [filecite\turn1file0\L11-L56](#)

2. Final Report

Ye project ka **clean, academic, presentation/thesis-oriented version** hai. Isme same SHIA-RAG idea ko structured research-paper style me explain kiya gaya hai:

Problem → Literature → Research Gap → Architecture → Layers → Novelty → Mathematics → Implementation → Evaluation → Risks → Future Work

Iska abstract clearly batata hai ki SHIA-RAG flat chunk-based RAG ko replace karke **concept-based Knowledge Forest** banata hai, jisme hierarchy aur semantic relations dono use hote hain. [filecite\turn1file1\L81-L91](#)

Meri current understanding of the project

Core idea in one line

Traditional RAG:

Document → Fixed Chunks → Embeddings → Top-k Chunks → LLM

SHIA-RAG:

Document → Concepts + Relations → Knowledge Forest → Hierarchical Retrieval → Context Optimization
→ Verified Answer → Feedback-based Evolution

Yahi project ka fundamental shift hai: **text chunks ko directly retrieve karne ke bajay concepts aur unke structured relationships ko retrieve karna.** [filecite\turn1file3\L174-L199](#)

Complete architecture: Layer 0 to Layer 8

Maine dono files ke according complete pipeline ko is tarah map kiya hai:

Layer	Kaam
Layer 0	Core Data Model
Layer 1	Input Processing
Layer 2	Document Parsing
Layer 3	Knowledge Extraction
Layer 4	Relation Management
Layer 5	SHIA Core / Hierarchy Construction

| **Layer 6** | Retrieval Engine |

| **Layer 7** | Generation + Verification |

| **Layer 8** | Operations + Feedback + Self-Evolution |

Ye architecture three major phases me divide hota hai:

- **Offline:** Layers 1–5 → documents se Knowledge Forest build karna
- **Online:** Layers 6–7 → query answer karna
- **Continuous:** Layer 8 → feedback se forest evolve karna

Layer 0 sabke neeche common foundation hai. `filecite turn1file1 L97-L109`

Most important thing: Complete Report is more corrected

Mujhe dono files me ye major distinction mila:

Final Report = clean specification

Complete Report = corrected + reconciled specification

Complete Report me specifically pehle ke documents ki problems identify karke fix ki gayi hain, jaise:

1. Cycle detection bug

Hierarchy me cycle detect karne wala traversal pehle wrong direction me tha. Correct logic me proposed parent se **upar ancestors ki taraf** traverse karke check kiya jata hai ki new node already ancestor to nahi hai.

`filecite turn1file10 L607-L625`

2. PRE → HV feedback problem

Agar PRE sirf ek best parent choose kare aur HV usko reject kar de, node orphan ho sakta tha. Corrected design me PRE **ranked list** deta hai aur HV next candidate try karta hai.

3. Confidence propagation ordering

Parent confidence par dependent child ko arbitrary order me process nahi kar sakte. Isliye corrected pipeline me **topological ordering** required hai.

4. Tree Knapsack dependency issue

Flat knapsack me child independently select ho sakta tha. Lekin hierarchy me context coherence ke liye child select karne par required parent context bhi select hona chahiye.

5. Tree vs Graph contradiction

Ye ek important conceptual correction hai:

- **Hierarchical edges** → acyclic forest

- **Semantic edges / cross-links** → cycles allowed

Aur corrected design me hierarchical structure ke liye **strict single-parent tree constraint** rakha gaya hai; additional associations semantic edges se handle hote hain. [filecite\turn1file5\L289-L302](#)

SHIA-RAG ki 5 main novelty contributions

Dono reports ka central novelty framework ye hai:

Novelty 1 — Precedence-Constrained Tree Knapsack

Context selection me normal top-k nahi.

Rule:

Agar child concept select hua hai, to required parent context bhi respect hona chahiye.

Ye hierarchy-aware context selection hai.

Novelty 2 — Self-Evolving Knowledge Forest

User feedback retrieval structure ko affect karta hai.

Simple flow:

Successful Retrieval → path weights increase

Failed Retrieval → path weights decrease / restructuring flag

Novelty 3 — Multi-Objective Energy Minimization

Hierarchy placement ko sirf similarity se decide nahi kiya jata.

Multiple factors optimize hote hain:

- semantic fit
- depth
- confidence
- ontological consistency

Complete Report iske corrected objective ko explicitly multi-objective energy minimization ke form me define karta hai. [filecite\turn1file11\L639-L656](#)

Novelty 4 — Adaptive Depth Traversal

Har query ke liye same depth retrieve nahi hoti.

For example:

- Definition → shallow
- Comparison → medium / multiple subtrees

- Multi-hop → deeper traversal
- Synthesis → broader retrieval

▯filecite▯turn1file15▯L903-L912▯

Novelty 5 — SHEF

Hierarchical RAG ke liye dedicated evaluation framework.

Isme sirf answer accuracy nahi, balki:

- Hop Accuracy
- Ancestor Recall
- Context Coherence
- Parent Assignment Accuracy
- Tree Edit Distance
- Forest Density
- Cross-document Fusion Accuracy
- Orphan Rate

jaise hierarchy-specific metrics bhi hain. ▯filecite▯turn1file15▯L916-L953▯

Sabse important architectural understanding jo maine note ki hai

SHIA-RAG actually 3 different things combine karta hai:

A. Knowledge Extraction System

Layers 1–4

Document ko parse karke:

Raw File → Text → Blocks → Concepts → Canonical Nodes → Validated Relations

B. Hierarchy Construction System

Layer 5

Main pipeline:

CPG++ → PRE → HV → FI → CLD

- **CPG++** = possible parents generate karta hai
- **PRE** = candidates rank karta hai
- **HV** = structural validation karta hai
- **FI** = valid node ko forest me insert karta hai
- **CLD** = additional semantic cross-links discover karta hai

C. Hierarchy-Aware RAG System

Layers 6–8

Query → Seed Nodes → Adaptive Traversal → Tree Knapsack → Context → LLM → Claim Verification → Feedback → Forest Update

Yahi full closed loop hai.

Dono files ko compare karne ke baad mera conclusion

Agar hume project ko study, implement, ya defend karna hai, to Complete Report ko primary technical reference maanna chahiye.

Final Report ko clean academic explanation/reference maana chahiye.

Kyunki Complete Report khud ko consolidated version ke roop me position karta hai aur usme Layer 0–8 ke corrected algorithms, missing schemas/functions, implementation details aur future steps consolidated hain.

□filecite□turn1file2□L135-L163□

Ab aage main tumhe is project ko exactly tumhare preferred style me padha sakta hoon:

Ek hi example ko poore project me carry karenge, aur ek time par sirf ek layer:

Example:

Operating Systems document → “Round Robin” concept

Har layer par main bataunga:

1. Layer ko exact input kya mila
2. Us input ka format kya hai
3. Layer internally kya-kya karta hai
4. Kaunsa algorithm kahan use hota hai
5. Output exactly kya banta hai
6. Output next layer ko kaise jata hai
7. Why ye step needed hai
8. Is layer me possible errors kya hain
9. Corrected report ke according kya safeguards hain

Main dono files ko base bana kar hi aage explain karunga—koi unsupported assumption nahi add karunga.

Agar tum bolo “Layer 0 se start karo”, to main ab Layer 0 ko ekdum detailed me, same Round Robin example ke saath start karta hoon.

YOU 19/08/2026, 22:48:03

1. Layer ko exact input kya mila
2. Us input ka format kya hai

3. Layer internally kya-kya karta hai
4. Kaunsa algorithm kahan use hota hai
5. Output exactly kya banta hai
6. Output next layer ko kaise jata hai
7. Why ye step needed hai
8. Is layer me possible errors kya hain
9. Corrected report ke according kya safeguards hain
10. exception handling kese ho rhi hai

ab tum ek example lo jisse me mujhe complete architure smjh me aaye
tum mujhe layer 6 se smjhao

CHATGPT 19/08/2026, 22:48:10

```
{"paths":["files"],"query":"search"}
```

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite{turn2file0}`

```
[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_0000000033088207b45d2ae58cbd406c
[L4] version_id: 1
[L5] name: SHIA_RAG_Complete_Report(8).md
[L6] mime_type: text/markdown
[L7] surface: conversation
[L8] score: 0.03200204813108039
[L9] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--45
[L10] snippet:
[L11] @dataclass
[L12] class RetrievalResult:
[L13] """Output of Layer 6 — input to Layer 7."""
[L14] selected_nodes: list # Nodes selected by Tree Knapsack
[L15] context_text: str # Assembled context string
[L16] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L17] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L18] query_intent: str # DEFINITION, COMPARISON, etc.
[L19] seed_node_ids: list # Initial seed nodes from vector search
[L20] token_count: int # Estimated tokens in context
[L21]
[L22] class RetrievalEngine:
[L23] def __init__(self, forest, vector_index, cache_manager, config):
[L24] self.forest = forest
[L25] self.vector_index = vector_index
[L26] self.cache = cache_manager # Redis cache integration
```

```
[L27] self.config = config
[L28] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L29]
[L30] def retrieve(self, query) -> RetrievalResult:
[L31] # Step 0: Cache check
[L32] cache_key = hash_query(query)
[L33] cached = self.cache.get_query_result(cache_key)
[L34] if cached:
[L35] return cached
[L36]
[L37] # Step 1: Analyze query
[L38] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L39] complexity = classify_complexity(intent) # Rule-based mapping
[L40]
[L41] # Step 2: Embed and find seed nodes
[L42] query_emb = embed_query(query) # Asymmetric embedding model
[L43] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L44]
[L45] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L46] depth_config = {
[L47] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L48] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L49] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L50] }
[L51] traversal = depth_config[complexity]
[L52]
[L53] # Step 4: Traverse forest from all seeds
[L54] candidate_nodes = set()
[L55] for seed in seed_nodes:
[L56] candidate_nodes |= self.traverse(seed, traversal)
[L57]
[L58] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L59] selected = self.multi_tree_knapsack(
[L60] candidate_nodes,
[L61] budget=self.config.token_budget
[L62] )
[L63]
[L64] # Step 6: Assemble context in semantic order
[L65] context_text = self.assemble_context(selected, intent)
[L66]
[L67] # Build result with traversal path for Layer 8 feedback
[L68] result = RetrievalResult(
[L69] selected_nodes=selected,
[L70] context_text=context_text,
```

```
[L71] traversal_path_edges=self._get_path_edges(selected),
[L72] query_complexity=complexity,
[L73] query_intent=intent,
[L74] seed_node_ids=[s.id for s in seed_nodes],
[L75] token_count=estimate_tokens(context_text)
[L76] )
[L77]
[L78] # Cache result
[L79] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L80]
[L81] return result
[L82]
[L83] def multi_tree_knapsack(self, candidate_nodes, budget):
[L84] """Apply Tree Knapsack per distinct tree, then merge results.
[L85]
[L86] For multi-seed queries, candidates may span multiple trees.
[L87] We optimize each tree independently, then merge within budget.
```

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file1

```
[L88] metadata:
[L89] query_result_index: 1
[L90] file_id: file_00000000f620821186e54e582d59731e
[L91] version_id: 1
[L92] name: SHIA_RAG_Final_Report(3).md
[L93] mime_type: text/markdown
[L94] surface: conversation
[L95] score: 0.031754032258064516
[L96] document_chunk_id: file_00000000f620821186e54e582d59731e--1--20
[L97] snippet:
[L98]
```



```

[L99] @dataclass
[L100] class RetrievalResult:
[L101]     """Output of Layer 6 – input to Layer 7."""
[L102]     selected_nodes: list          # Nodes selected by Tree Knapsack
[L103]     context_text: str            # Assembled context string
[L104]     traversal_path_edges: list    # Edges traversed (for Layer 8 feedback)
[L105]     query_complexity: str         # SIMPLE, MODERATE, COMPLEX
[L106]     query_intent: str            # DEFINITION, COMPARISON, etc.
[L107]     seed_node_ids: list          # Initial seed nodes from vector search
[L108]     token_count: int             # Estimated tokens in context
[L109]
[L110] class RetrievalEngine:
[L111]     def __init__(self, forest, vector_index, cache_manager, config):
[L112]         self.forest = forest
[L113]         self.vector_index = vector_index
[L114]         self.cache = cache_manager
[L115]         self.config = config
[L116]
[L117]     def retrieve(self, query) -> RetrievalResult:
[L118]         # Step 0: Cache check
[L119]         cached = self.cache.get_query_result(hash_query(query))
[L120]         if cached:
[L121]             return cached
[L122]
[L123]         # Step 1: Classify query intent and complexity
[L124]         intent = detect_intent(query)
[L125]         complexity = classify_complexity(intent)
[L126]
[L127]         # Step 2: Embed query and find seed nodes
[L128]         query_emb = embed_query(query) # Asymmetric embedding model
[L129]         seed_nodes = self.vector_index.search(query_emb, k=5)
[L130]
[L131]         # Step 3: Adaptive depth traversal
[L132]         depth_config = {
[L133]             'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L134]             'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L135]             'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True}
[L136]         }
[L137]         traversal = depth_config[complexity]
[L138]
[L139]         # Step 4: BFS traversal from all seeds
[L140]         candidate_nodes = set()
[L141]         for seed in seed_nodes:
[L142]             candidate_nodes |= self.traverse(seed, traversal)
[L143]
[L144]         # Step 5: Tree Knapsack selection (per-tree, then merge)
[L145]         selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L146]
[L147]         # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L148]         context_text = self.assemble_context(selected, intent)
[L149]
[L150]         return RetrievalResult(
[L151]             selected_nodes=selected, context_text=context_text,
[L152]             traversal_path_edges=self._get_path_edges(selected),
[L153]             query_complexity=complexity, query_intent=intent,
[L154]             seed_node_ids=[s.id for s in seed_nodes],
[L155]             token_count=estimate_tokens(context_text)
[L156]         )
[L157]

```

[L158]

[L159] ##### Multi-Tree Knapsack Selection

[L160]

[L161] For multi-seed queries, candidates may span multiple trees. The algorithm:

[L162]

[L163] 1. **Group** candidates by tree root

[L164] 2. **Allocate** token budget proportionally to each tree

[L165] 3. **Run** Tree Knapsack on each tree independently

[L166] 4. **Merge** results and trim to total budget

[L167]

[L168] ```python

[L169] def multi_tree_knapsack(self, candidate_nodes, budget):

[L170] tree_groups = {}

[L171] for node in candidate_nodes:

[L172] root = self.forest.find_root(node)

[L173] tree_groups.setdefault(root.id, []).append(node)

[L174]

[L175] all_selected = []

[L176] for root_id, nodes in tree_groups.items():

[L177] tree_budget = int(budget * len(nodes) / max(len(candidate_nodes), 1))

[L178] _, selected = tree_knapsack(self.forest.get_node(root_id), tree_budget, self.

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file2

[L179] metadata:

[L180] query_result_index: 2

[L181] file_id: file_00000000f620821186e54e582d59731e

[L182] version_id: 1

[L183] name: SHIA_RAG_Final_Report(3).md

[L184] mime_type: text/markdown

[L185] surface: conversation

[L186] score: 0.0315136476426799

[L187] document_chunk_id: file_00000000f620821186e54e582d59731e--1--19

[L188] snippet:

[L189] ##### 5.5.5 CLD — Cross-Link Discoverer

[L190]

[L191] After hierarchical placement, discover semantic (non-hierarchical) links:

[L192]

[L193]

```

[L194] def cld_discover(node, forest, config):
[L195]     """Find semantic cross-links for a newly placed node."""
[L196]     cross_links = []
[L197]     neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L198]
[L199]     for neighbor in neighbors:
[L200]         if neighbor.id == node.id or neighbor.id == node.parent:
[L201]             continue
[L202]
[L203]         sim = cosine_similarity(node.embedding, neighbor.embedding)
[L204]         if sim > config.cross_link_threshold:
[L205]             relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L206]             if relation_type:
[L207]                 forest.add_edge(node.id, neighbor.id,
[L208]                     edge_class='SEMANTIC', relation_type=relation_type,
[L209]                     confidence=sim, weight=1.0)
[L210]                 cross_links.append((neighbor.id, relation_type, sim))
[L211]
[L212]     return cross_links
[L213]

```

[L214]

[L215] ---

[L216]

[L217] ## 6. Online Pipeline — Layers 6-7

[L218]

[L219] ### 6.1 Layer 6: Retrieval Engine

[L220]

[L221] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.

[L222]

[L223] **Input:** User query string → **Output:** RetrievalResult

[L224]

[L225] ##### Query Complexity Classification

[L226]

[L227] | Intent | Example | Complexity | Traversal Depth |

[L228] |-----|-----|-----|-----|

[L229] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |

[L230] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |

[L231] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |

[L232] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |

[L233] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |

[L234]

[L235] ##### Retrieval Pipeline

[L236]

[L237] ```python

[L238] @dataclass

[L239] class RetrievalResult:

[L240] """Output of Layer 6 — input to Layer 7."""

[L241] selected_nodes: list # Nodes selected by Tree Knapsack

```

[L242] context_text: str # Assembled context string
[L243] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L244] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L245] query_intent: str # DEFINITION, COMPARISON, etc.
[L246] seed_node_ids: list # Initial seed nodes from vector search
[L247] token_count: int # Estimated tokens in context
[L248]
[L249] class RetrievalEngine:
[L250] def __init__(self, forest, vector_index, cache_manager, config):
[L251] self.forest = forest
[L252] self.vector_index = vector_index
[L253] self.cache = cache_manager
[L254] self.config = config
[L255]
[L256] def retrieve(self, query) -> RetrievalResult:
[L257] # Step 0: Cache check
[L258] cached = self.cache.get_query_result(hash_query(query))
[L259] if cached:
[L260] return cached
[L261]
[L262] # Step 1: Classify query intent and complexity
[L263] intent = detect_intent(query)
[L264] complexity = classify_complexity(intent)
[L265]
[L266] # Step 2: Embed query and find seed nodes
[L267] query_emb = embed_query(query) # Asymmetric embedding model
[L268] seed_nodes = self.vector_index.search(query_emb, k=5)
[L269]
[L270] # Step 3: Adaptive depth traversal
[L271] depth_config = {
[L272] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L273] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L274] 'COMPLEX': {'max_depth': 3, 'parents': True,

```

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file3

```

[L275] metadata:
[L276] query_result_index: 3
[L277] file_id: file_0000000033088207b45d2ae58cbd406c
[L278] version_id: 1
[L279] name: SHIA_RAG_Complete_Report(8).md
[L280] mime_type: text/markdown
[L281] surface: conversation
[L282] score: 0.03047794966520434

```

```

[L283] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--63
[L284] snippet:
[L285] 5"""
[L286] "down": """ALTER TABLE knowledge_nodes
[L287] DROP COLUMN abstraction_level,
[L288] DROP COLUMN base_confidence,
[L289] DROP COLUMN extraction_confidence"""
[L290] },
[L291] {
[L292] "version": 3,
[L293] "description": "Migrate properties from string array to JSONB",
[L294] "up": """ALTER TABLE knowledge_nodes
[L295] ALTER COLUMN data TYPE JSONB USING data::JSONB""",
[L296] "down": "-- Manual rollback required"
[L297] }
[L298] ]
[L299]
[L300] def run_migrations(db, target_version=None):
[L301] current = db.get_migration_version()
[L302] for migration in MIGRATIONS:
[L303] if migration["version"] > current:
[L304] if target_version and migration["version"] > target_version:
[L305] break
[L306] db.execute(migration["up"])
[L307] db.set_migration_version(migration["version"])
[L308] print(f"Applied migration v{migration['version']}: {migration['description']}")
[L309]

```

```

[L310]
[L311] ---
[L312]
[L313] ### 7.13 Error Handling & Retry Logic
[L314]
[L315]

```

python

```

[L316] import time
[L317] from functools import wraps
[L318]
[L319] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L320] """Decorator for retrying failed operations with exponential backoff."""
[L321] def decorator(func):
[L322] @wraps(func)
[L323] def wrapper(*args, **kwargs):
[L324] for attempt in range(max_retries):
[L325] try:
[L326] return func(*args, **kwargs)

```

```

[L327] except (TimeoutError, ConnectionError) as e:
[L328] if attempt == max_retries - 1:
[L329] raise
[L330] delay = min(base_delay * (2 ** attempt), max_delay)
[L331] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L332] time.sleep(delay)
[L333] return wrapper
[L334] return decorator
[L335]
[L336] class ProcessingPipeline:
[L337] """Error handling for the SHIA-RAG processing pipeline."""
[L338]
[L339] def process_document(self, doc):
[L340] # Layer 1: OCR with fallback
[L341] try:
[L342] text = self.extract_text(doc)
[L343] except OCRFailure:
[L344] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L345]
[L346] # Layer 3: LLM extraction with retry + fallback model
[L347] try:
[L348] concepts = self.llm_extract_concepts(text)
[L349] except LLMTimeout:
[L350] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L351]
[L352] # Layer 6: Vector DB with cache fallback
[L353] try:
[L354] results = self.vector_search(query_embedding)
[L355] except VectorDBUnavailable:
[L356] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L357]
[L358] return {"status": "SUCCESS", "concepts": concepts}
[L359]

```

```

[L360]
[L361] ---
[L362]
[L363] ### 7.14 Caching Strategy
[L364]
[L365]

```

python

```

[L366] import redis
[L367] import hashlib
[L368] import json
[L369]
[L370] class CacheManager:

```

```
[L371] """Multi-layer caching for SHIA-RAG."""
[L372]
[L373] def __init__(self, redis_url="redis://localhost:6379"):
[L374] self.redis = redis.from_url(redis_url)
[L375] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L376] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L377] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L378]
[L379] def get_query_result(self, query_text):
[L380] """Check if this exact query has been answered recently."""
```

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file4

```
[L381] metadata:
[L382] query_result_index: 4
[L383] file_id: file_0000000033088207b45d2ae58cbd406c
[L384] version_id: 1
[L385] name: SHIA_RAG_Complete_Report(8).md
[L386] mime_type: text/markdown
[L387] surface: conversation
[L388] score: 0.029726775956284153
[L389] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--83
[L390] snippet:
[L391] idempotent operations |
[L392]
[L393] ---
[L394]
[L395] ## 12. Conclusion
[L396]
[L397] ### 12.1 Summary of Work Done
[L398]
[L399] 1. Read and analyzed all 6 source PDFs + 914-page audit (~2M+ characters total)
[L400] 2. Identified and fixed 75 issues across all 9 layers (Layer 0–8), including 38 critical bugs, 25 important
gaps, 7 ambiguities
[L401] 3. Fixed all formula contradictions: PRE depth formula unified to  $(D_{\max} - \text{depth}) / D_{\max}$ , self-evolution
feedback formula unified to  $\exp(\eta \cdot f)$ 
[L402] 4. Resolved cross-document inconsistencies (3 PRE formulas → 1, 13 modules → 8 layers, single-
parent constraint declared)
[L403] 5. Added 5 novel contributions with literature search methodology and novelty level assessment
[L404] 6. Designed complete implementation architecture with folder structure, tech stack, .gitignore,
CI/CD
[L405] 7. Created evaluation framework (SHEF) with metrics, datasets, baselines, statistical significance
testing, error analysis, and computational cost analysis
[L406] 8. Planned 6-month implementation roadmap with clear milestones
```

- [L407] 9. **Added detailed sections for ALL layers** (Layers 1–4 were previously missing dedicated sections)
- [L408] 10. **Added Layer 7 complete implementation:** prompt template, claim extraction, evidence verification, citation binding, confidence formula, unsupported/partial/empty-context policies, structured output contract
- [L409] 11. **Added Layer 8 complete implementation:** logging schema, feedback acquisition, feedback attribution isolation, self-evolution loop closure (edge weights → utility() → Tree Knapsack), monitoring metrics, weight normalization/decay
- [L410] 12. **Defined all previously undefined functions:** `flag_for_restructure()`, `rebalance_forest()`, `verify_claim()`, `compute_answer_confidence()`, `format_citation()`, `assemble_context()`, `multi_tree_knapsack()`, `layer1_process()`, `layer2_parse()`, `layer3_extract()`, `layer4_relate()`, `alias_resolution()`
- [L411] 13. **Added missing schemas:** `KnowledgeUnit`, `NormalizedDocument`, `DocumentBlock`, `RetrievalResult`, `GenerationResult`, `VerifiedClaim`, `QueryEvent`, `QueryLog`
- [L412] 14. **Defined complete relation ontology:** 12 relation types across hierarchical (IS_A, PART_OF, SUBCLASS_OF, INSTANCE_OF) and semantic (USES, CAUSES, DEPENDS_ON, CONTRASTS_WITH, SIMILAR_TO, HAS_PROPERTY, PRODUCES, PREREQUISITE) categories
- [L413]
- [L414] **12.2 Key Novelties (Paper-Worthy)**
- [L415]
- [L416] | # | Novelty | Level | Why It's New |
- [L417] |---|-----|-----|-----|
- [L418] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |
- [L419] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |
- [L420] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |
- [L421] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |
- [L422] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |
- [L423]
- [L424] **12.3 Next Steps**

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file5

- [L425] metadata:
- [L426] query_result_index: 5
- [L427] file_id: file_00000000f620821186e54e582d59731e
- [L428] version_id: 1
- [L429] name: SHIA_RAG_Final_Report(3).md
- [L430] mime_type: text/markdown
- [L431] surface: conversation
- [L432] score: 0.029571646010002173

[L433] document_chunk_id: file_00000000f620821186e54e582d59731e--1--6

[L434] snippet:

[L435]

```
[L436] graph TD
[L437]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L438]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Preprocessing<br/>Tokenization, embedding"]
[L439]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L440]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L441]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L442]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L443]     end
[L444]
[L445]     subgraph "Phase 2: Online – Retrieval & Generation"
[L446]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, LSH"]
[L447]         KF --> L6
[L448]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L449]         L7 --> A["Answer + Citations"]
[L450]     end
[L451]
[L452]     subgraph "Phase 3: Continuous – Operations"
[L453]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L454]         A --> L8
[L455]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L456]     end
[L457]
```

[L458]

[L459] ### 3.2 Data Flow

[L460]

[L461]

```
[L462] Raw Document (PDF/DOCX/HTML)
[L463]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L464] Normalized Text + Metadata
[L465]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L466] [Page → Region → Block] hierarchy
[L467]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L468] KnowledgeUnit objects (candidate concepts)
[L469]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L470] KnowledgeNode objects (canonical, deduplicated)
[L471]     ↓ Layer 4: Validate relations against ontology, score confidence
[L472] KnowledgeEdge objects (typed, scored)
[L473]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L474] Knowledge Forest (rooted trees + cross-links)
[L475]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L476] Selected context nodes (within token budget)
[L477]     ↓ Layer 7: Build prompt, invoke LLM
[L478]     ↓ Layer 7: Verify claims against evidence, bind citations
[L479] Answer with citations + confidence score
[L480]     ↓ Layer 8: Log, collect feedback, update forest weights
[L481]
```

[L482]

[L483] ### 3.3 Layer Summary

[L484]

[L485] | Layer | Name | Input | Output | Key Algorithms |
[L486] |-----|-----|-----|-----|-----|
[L487] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L488] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L489] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L490] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L491] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L492] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L493] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L494] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L495] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L496]
[L497] ---
[L498]
[L499] ## 4. Data Model — Layer 0

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file6

[L500] metadata:
[L501] query_result_index: 6
[L502] file_id: file_0000000033088207b45d2ae58cbd406c
[L503] version_id: 1
[L504] name: SHIA_RAG_Complete_Report(8).md
[L505] mime_type: text/markdown
[L506] surface: conversation
[L507] score: 0.02904040404040404
[L508] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--1
[L509] snippet:
[L510] 6 Layer 5: SHIA Core](#76-layer-5-shia-core-corrected-pipeline)
[L511] - 7.7 Layer 6: Retrieval Engine
[L512] - 7.8 Layer 7: Generation Engine
[L513] - 7.9 Layer 8: Operations & Self-Evolution
[L514] - 7.10 Schema Validation
[L515] - 7.11 Database Indexing
[L516] - 7.12 Migration Strategy
[L517] - 7.13 Error Handling
[L518] - 7.14 Caching Strategy
[L519] - 7.15 Monitoring & Alerting
[L520] - 7.16 Multi-Tenancy
[L521] - 7.17 Async Processing
[L522] 8. Mathematical Formulations (Corrected)
[L523] 9. Implementation Roadmap
[L524] 10. Evaluation Framework

[L525] - 10.4 Statistical Significance

[L526] - 10.5 Error Analysis

[L527] - 10.6 Computational Cost

[L528] 11. Risk Analysis & Mitigation

[L529] 12. Conclusion

[L530]

[L531] ---

[L532]

[L533] ## 1. Project Overview

[L534]

[L535] ### 1.1 What is SHIA-RAG?

[L536]

[L537] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) is a next-generation RAG system that fundamentally changes how information is retrieved for LLM-based question answering.

[L538]

[L539] **Traditional RAG** splits documents into flat text chunks → embeds them → retrieves top-k similar chunks → feeds to LLM.

[L540]

[L541] **SHIA-RAG** instead:

[L542] 1. **Extracts knowledge** (concepts, definitions, relations) from documents

[L543] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes (not text chunks)

[L544] 3. **Retrieves knowledge nodes** by traversing the hierarchy (parents, children, siblings)

[L545] 4. **Assembles context** intelligently within a token budget

[L546] 5. **Generates answers** with citation-backed evidence

[L547]

[L548] ### 1.2 Why It Matters

[L549]

[L550] | Problem with Traditional RAG | How SHIA-RAG Solves It |

[L551] |---|

[L552] | Chunks break semantic continuity | Knowledge nodes preserve complete concepts |

[L553] | Duplicate/fragmented information | Entity resolution merges synonyms into single nodes |

[L554] | Fixed chunk granularity | Hierarchy allows multi-level retrieval (broad → specific) |

[L555] | Weak multi-hop reasoning | Parent-child paths encode reasoning chains |

[L556] | No cross-document fusion | Knowledge Forest merges concepts across all documents |

[L557]

[L558] ### 1.3 Core Innovation

[L559]

[L560] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "CS → OS → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L561]

[L562] ---

[L563]

[L564] ## 2. Document Inventory & Summary

[L565]
[L566] ### 2.1 All 6 PDFs Analyzed
[L567]
[L568] | # | File Name | Pages | Content Type | What It Contains |
[L569] |---|-----|-----|-----|-----|
[L570] | 1 | `first Pdf.

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file7

[L571] metadata:
[L572] query_result_index: 7
[L573] file_id: file_0000000033088207b45d2ae58cbd406c
[L574] version_id: 1
[L575] name: SHIA_RAG_Complete_Report(8).md
[L576] mime_type: text/markdown
[L577] surface: conversation
[L578] score: 0.02877846790890269
[L579] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--64
[L580] snippet:
[L581] class ProcessingPipeline:
[L582] """Error handling for the SHIA-RAG processing pipeline."""
[L583]
[L584] def process_document(self, doc):
[L585] # Layer 1: OCR with fallback
[L586] try:
[L587] text = self.extract_text(doc)
[L588] except OCRFailure:
[L589] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L590]
[L591] # Layer 3: LLM extraction with retry + fallback model
[L592] try:
[L593] concepts = self.llm_extract_concepts(text)
[L594] except LLMLTimeout:
[L595] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L596]
[L597] # Layer 6: Vector DB with cache fallback
[L598] try:
[L599] results = self.vector_search(query_embedding)
[L600] except VectorDBUnavailable:
[L601] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L602]
[L603] return {"status": "SUCCESS", "concepts": concepts}
[L604]

```
[L605]
[L606] ---
[L607]
[L608] ### 7.14 Caching Strategy
[L609]
[L610]
```

python

```
[L611] import redis
[L612] import hashlib
[L613] import json
[L614]
[L615] class CacheManager:
[L616] """Multi-layer caching for SHIA-RAG."""
[L617]
[L618] def __init__(self, redis_url="redis://localhost:6379"):
[L619] self.redis = redis.from_url(redis_url)
[L620] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L621] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L622] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L623]
[L624] def get_query_result(self, query_text):
[L625] """Check if this exact query has been answered recently."""
[L626] key = f"query:{hashlib.sha256(query_text.encode()).hexdigest()}"
[L627] cached = self.redis.get(key)
[L628] return json.loads(cached) if cached else None
[L629]
[L630] def cache_query_result(self, query_text, result):
[L631] key = f"query:{hashlib.sha256(query_text.encode()).hexdigest()}"
[L632] self.redis.setex(key, self.QUERY_CACHE_TTL, json.dumps(result))
[L633]
[L634] def get_node(self, node_id):
[L635] """Cache frequently accessed nodes."""
[L636] key = f"node:{node_id}"
[L637] cached = self.redis.get(key)
[L638] return json.loads(cached) if cached else None
[L639]
[L640] def cache_node(self, node_id, node_data):
[L641] key = f"node:{node_id}"
[L642] self.redis.setex(key, self.NODE_CACHE_TTL, json.dumps(node_data))
[L643]
```

```
[L644]
[L645] ---
[L646]
[L647] ### 7.15 Monitoring & Alerting
[L648]
[L649]
```

text

[L650] Monitoring Stack:
[L651] - Metrics: Prometheus (via prometheus_client Python library)
[L652] - Dashboards: Grafana
[L653] - Logs: ELK Stack (Elasticsearch + Logstash + Kibana)
[L654] - Alerts: Grafana Alerts → Slack/Email
[L655]
[L656] Key Metrics:
[L657]

[L658]	Metric	Alert Threshold
[L659]		
[L660]	Query latency P50/P95/P99	P95 > 2s → WARNING, P99 > 5s → ALERT
[L661]	Error rate (layer-wise)	> 5% in any layer → ALERT
[L662]	Cache hit ratio	< 30% → WARNING
[L663]	LLM API error rate	> 10% → ALERT
[L664]	Forest node count	> 1M → capacity planning WARNING
[L665]	Orphan rate	> 20% → hierarchy quality WARNING
[L666]	Feedback ratio (helpful)	< 60% → retrieval quality WARNING
[L667]		
[L668]	````	
[L669]		
[L670]	---	

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file8

[L671] metadata:
[L672] query_result_index: 8
[L673] file_id: file_0000000033088207b45d2ae58cbd406c
[L674] version_id: 1
[L675] name: SHIA_RAG_Complete_Report(8).md
[L676] mime_type: text/markdown
[L677] surface: conversation
[L678] score: 0.028531243721117136
[L679] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--19
[L680] snippet:
[L681] ---
[L682]
[L683] ### Novelty 3: Multi-Objective Hierarchy Energy Minimization
[L684]
[L685] **What exists:** TreeRAG and RAPTOR use simple clustering. GraphRAG uses flat triple extraction.
[L686]

[L687] **What SHIA-RAG proposes:** Hierarchy construction as a formal **energy minimization** problem:

[L688]

[L689]
$$J(\mathcal{F}) = \sum_{(P_i, N_i)} \left[\lambda_1 \cdot L_{\text{sem}}(P_i, N_i) + \lambda_2 \cdot L_{\text{depth}}(N_i) + \lambda_3 \cdot L_{\text{conf}}(P_i, N_i) + \lambda_4 \cdot L_{\text{onto}}(P_i, N_i) \right]$$

[L690]

[L691] Where:

[L692] - L_{sem} = semantic distance loss (1 - cosine similarity)

[L693] - L_{depth} = depth regularization (penalizes too-deep or too-shallow trees)

[L694] - L_{conf} = confidence loss (low-confidence placements increase energy)

[L695] - L_{onto} = ontological consistency loss (parent should be more general)

[L696]

[L697] **Why novel:** No RAG paper has formalized hierarchy construction as a multi-objective optimization problem for hierarchy induction in RAG. The formulation enables principled trade-offs between semantic fit, structural balance, confidence, and ontological consistency.

[L698]

[L699] > [!NOTE]

[L700] > The L_{onto} term uses a **sigmoid relaxation** $\sigma(\text{type_score}(P) - \text{type_score}(N))$ instead of a non-differentiable indicator function. See Section 8.1 for the corrected formulation.

[L701]

[L702] ---

[L703]

[L704] ### Novelty 4: Adaptive Depth Traversal via Query Complexity Classification

[L705]

[L706] **What exists:** All RAG systems retrieve a fixed number of chunks/nodes regardless of query complexity.

[L707]

[L708] **What SHIA-RAG proposes:** A **query complexity classifier** that dynamically adjusts traversal depth:

[L709]

[L710] | Query Type | Example | Traversal Strategy |

[L711] |-----|-----|-----|

[L712] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |

[L713] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |

[L714] | MULTI-HOP | "How does OSPF convergence affect network reliability?" | Deep: Full path traversal + cross-links (depth=3+) |

[L715] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Retrieve at root/topic level, expand selectively |

[L716]

[L717] ```python

[L718] class QueryComplexityClassifier:

[L719] def classify(self, query):

[L720] features = {

[L721] 'num_entities': count_entities(query),

[L722] 'has_comparison': any(w in query.lower() for w in ['compare', 'vs', 'difference']),

[L723] 'has_causal': any(w in query.lower() for w in ['why', 'how', 'cause', 'effect']),

```
[L724] 'num_hops_estimated': estimate_reasoning_hops(query),
[L725] }
[L726] # Lightweight classifier (fine-tuned BERT or rule-based)
[L727] return self.model.predict(features) # → SIMPLE | MODERATE | COMPLEX | SYNTHESIS
[L728]
[L729] def estimate_reasoning_hops(query):
[L730] """Rule-based heuristic to estimate the number of reasoning hops
[L731] required to answer a query. Returns an integer in [1, 5]."""
```

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file9

```
[L732] metadata:
[L733] query_result_index: 9
[L734] file_id: file_0000000033088207b45d2ae58cbd406c
[L735] version_id: 1
[L736] name: SHIA_RAG_Complete_Report(8).md
[L737] mime_type: text/markdown
[L738] surface: conversation
[L739] score: 0.028309409888357256
[L740] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--20
[L741] snippet:
[L742] ---
[L743]
[L744] ### Novelty 4: Adaptive Depth Traversal via Query Complexity Classification
[L745]
[L746] What exists: All RAG systems retrieve a fixed number of chunks/nodes regardless of query complexity.
[L747]
[L748] What SHIA-RAG proposes: A query complexity classifier that dynamically adjusts traversal depth:
[L749]
[L750] | Query Type | Example | Traversal Strategy |
[L751] |-----|-----|-----|
[L752] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |
[L753] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |
[L754] | MULTI-HOP | "How does OSPF convergence affect network reliability?" | Deep: Full path traversal +
cross-links (depth=3+) |
[L755] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Retrieve at root/topic level, expand selectively
|
[L756]
[L757]
```



```

[L758] class QueryComplexityClassifier:
[L759]     def classify(self, query):
[L760]         features = {
[L761]             'num_entities': count_entities(query),
[L762]             'has_comparison': any(w in query.lower() for w in ['compare', 'vs', 'difference']),
[L763]             'has_causal': any(w in query.lower() for w in ['why', 'how', 'cause', 'effect']),
[L764]             'num_hops_estimated': estimate_reasoning_hops(query),
[L765]         }
[L766]         # Lightweight classifier (fine-tuned BERT or rule-based)
[L767]         return self.model.predict(features) # → SIMPLE | MODERATE | COMPLEX | SYNTHESIS
[L768]
[L769] def estimate_reasoning_hops(query):
[L770]     """Rule-based heuristic to estimate the number of reasoning hops
[L771]     required to answer a query. Returns an integer in [1, 5]."""
[L772]     hops = 1
[L773]
[L774]     # Causal/explanatory queries need more hops
[L775]     if any(w in query.lower() for w in ['how', 'why', 'explain']):
[L776]         hops += 1
[L777]
[L778]     # Impact/effect queries need cross-concept reasoning
[L779]     if any(w in query.lower() for w in ['affect', 'impact', 'effect', 'influence']):
[L780]         hops += 1
[L781]
[L782]     # Conjunctions suggest multi-concept queries
[L783]     if any(w in query.lower() for w in [' and ', ' or ', ' both ', ' versus ']):
[L784]         hops += 1
[L785]
[L786]     # Comparison queries need sibling traversal
[L787]     if any(w in query.lower() for w in ['compare', 'vs', 'difference', 'similar']):
[L788]         hops += 1
[L789]
[L790]     return min(hops, 5)
[L791]

```

[L792]

[L793] **Why novel:** Dynamic retrieval depth adaptation based on query complexity has not been proposed in any hierarchical RAG paper.

[L794]

[L795] ---

[L796]

[L797] **### Novelty 5: SHEF — SHIA-RAG Evaluation Framework**

[L798]

[L799] **What exists:** Standard RAG benchmarks (MS MARCO, NaturalQuestions) only measure flat retrieval. No benchmark tests hierarchical retrieval quality.

[L800]

[L801] **What SHIA-RAG proposes:** A novel evaluation framework with **3 metric categories**:

[L802]

[L803] **#### A. Retrieval Metrics (RM)**

[L804] | Metric | Formula | What It Measures |

[L805] |-----|-----|-----|

[L806] | Hop Accuracy | $\frac{\text{correct multi-hop answers}}{\text{total multi-hop queries}}$ | Multi-hop reasoning quality |

[L807] | Ancestor Recall | $\frac{|\text{retrieved ancestors} \cap \text{gold ancestors}|}{|\text{gold ancestors}|}$ | Whether the system retrieves the correct concept chain |

[L808] | Context Coherence | Avg semantic similarity between consecutive nodes in context | Whether assembled context makes logical sense |

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite[turn2file10]`

[L809] metadata:

[L810] query_result_index: 10

[L811] file_id: file_00000000f620821186e54e582d59731e

[L812] version_id: 1

[L813] name: SHIA_RAG_Final_Report(3).md

[L814] mime_type: text/markdown

[L815] surface: conversation

[L816] score: 0.02777777777777776

[L817] document_chunk_id: file_00000000f620821186e54e582d59731e--1--2

[L818] snippet:

[L819] | Problem | Description | Impact |

[L820] |-----|-----|-----|

[L821] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L822] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L823] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L824] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |

[L825] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L826]

[L827] ### 1.2 Proposed Solution: SHIA-RAG

[L828]

[L829] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L830]

[L831]

[L832] Traditional RAG:

Document → [Chunk₁, Chunk₂, ..., Chunk_n] → Top-k → LLM

[L833] SHIA-RAG:

Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM

[L834]

[L835]

[L836] The system:

[L837] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L838] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L839] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L840] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L841] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L842] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L843]

[L844] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L845]

[L846] ### 1.3 Objectives

[L847]

[L848] 1. Design a complete 8-layer architecture for hierarchical RAG

[L849] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L850] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L851] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L852] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L853] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L854]

[L855] ### 1.4 Contributions

[L856]

[L857] This work makes five novel contributions to the RAG literature:

[L858]

[L859] | # | Contribution | Novelty Level |

[L860] |---|-----|-----|

[L861] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L862] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L863] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L864] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L865] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L866]

[L867] ---

[L868]

[L869] ## 2. Literature Review

[L870]

[L871] ### 2.1 Related Work

[L872]

[L873] ##### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 19/08/2026, 22:48:16

Citation Marker: `filecite`turn2file11

[L874] metadata:

[L875] query_result_index: 11

[L876] file_id: file_0000000033088207b45d2ae58cbd406c

[L877] version_id: 1

[L878] name: SHIA_RAG_Complete_Report(8).md

[L879] mime_type: text/markdown

[L880] surface: conversation

[L881] score: 0.027623285131627734

[L882] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--84

[L883] snippet:

[L884]  **Defined all previously undefined functions:** `flag_for_restructure()` , `rebalance_forest()` , `verify_claim()` , `compute_answer_confidence()` , `format_citation()` , `assemble_context()` , `multi_tree_knapsack()` , `layer1_process()` , `layer2_parse()` , `layer3_extract()` , `layer4_relate()` , `alias_resolution()`

[L885] 13.  **Added missing schemas:** `KnowledgeUnit` , `NormalizedDocument` , `DocumentBlock` , `RetrievalResult` , `GenerationResult` , `VerifiedClaim` , `QueryEvent` , `QueryLog`

[L886] 14.  **Defined complete relation ontology:** 12 relation types across hierarchical (IS_A, PART_OF, SUBCLASS_OF, INSTANCE_OF) and semantic (USES, CAUSES, DEPENDS_ON, CONTRASTS_WITH, SIMILAR_TO, HAS_PROPERTY, PRODUCES, PREREQUISITE) categories

[L887]

[L888] ### 12.2 Key Novelties (Paper-Worthy)

[L889]

[L890] | # | Novelty | Level | Why It's New |

[L891] |---|-----|-----|-----|

[L892] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L893] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L894] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L895] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L896] | 5 | **SHF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L897]

[L898] ### 12.3 Next Steps

[L899]

[L900] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)[L901] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)[L902] 3. **Build Layers 1–2** (Document ingestion and parsing pipeline)[L903] 4. **Build Layer 3** (Knowledge extraction with LLM)[L904] 5. **Build Layer 4** (Relation management with ontology validation)

[L905] 6. **Build Layer 5** (SHIA Core with corrected algorithms)
 [L906] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
 [L907] 8. **Build Layer 7** (Generation with verification)
 [L908] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
 [L909] 10. **Run first evaluation** on OS textbook dataset using SHEF
 [L910] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
 [L911] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
 [L912]
 [L913] ### 12.4 Future Work
 [L914]
 [L915] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)
 [L916] - **Multi-Tenancy**: Implement access control and document-level permissions
 [L917] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets
 [L918] - **Cross-Lingual Support**: Extend to multilingual documents and concept hierarchies
 [L919] - **Incremental Updates**: Support adding new documents without full forest reconstruction
 [L920] - **Advanced Verification**: Replace semantic similarity with NLI (Natural Language Inference) model for claim verification
 [L921] - **Exploration in Self-Evolution**: Add ϵ -greedy or Thompson sampling to edge weight updates to avoid exploitation-only bias
 [L922]
 [L923] ---

CHATGPT 19/08/2026, 22:48:16

Bilkul. Ab hum **sirf Layer 6** ko ekdum depth me samjhenge, aur tumhare diye hue **10 points** ko exactly follow karenge.

Main **same example** use karunga:

Knowledge Forest domain: Operating Systems

User Query: "How does Round Robin improve fairness and what is its effect on response time?"

Ye query achha example hai kyunki isme:

- ek target concept hai → **Round Robin**
- ek effect/relation hai → **fairness**
- dusra effect hai → **response time**
- query ko ek se zyada related concepts chahiye

Isliye hum Layer 6 ka complete behavior dekh sakte hain.

Layer 6 — Retrieval Engine

Sabse pehle: Layer 6 ka actual kaam kya hai?

Layer 6 ka kaam **answer generate karna nahi hai**.

Uska kaam hai:

User ke question ko samajhna → Knowledge Forest me relevant starting concepts dhundhna → hierarchy traverse karna → token budget ke andar best context select karna → Layer 7 ko structured RetrievalResult dena.

Report ke according Layer 6 ki responsibility exactly query analyze karna, Knowledge Forest traverse karna, aur token budget ke andar optimal context nodes select karna hai. [filecite\turn2file2\L217-L233](#)

Overall flow:

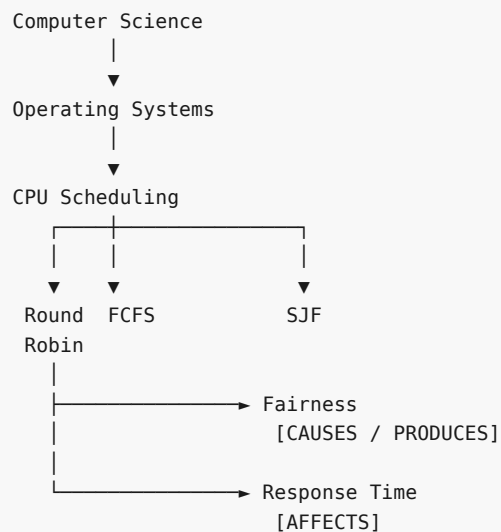
```
graph TD
    A[User Query] --> B[0] Cache Check
    B --> C[1] Intent + Complexity Analysis
    C --> D[2] Query Embedding
    D --> E[3] Seed Node Search
    E --> F[4] Adaptive Forest Traversal
    F --> G[5] Multi-Tree Knapsack
    G --> H[6] Context Assembly
    H --> I[RetrievalResult]
    I --> J[Layer 7]
```

Ye complete flow reports me specified hai. [filecite\turn2file0\L22-L81](#)

Pehle hum Layer 6 start hone se pehle ka environment samjhenge

Layer 5 already Knowledge Forest bana chuka hai.

Hamare example me simplified forest maan lo:



Important distinction:

```
Vertical links
Round Robin → CPU Scheduling → OS
              = HIERARCHICAL PATH

Side links
Round Robin → Fairness
Round Robin → Response Time
              = SEMANTIC CROSS-LINKS
```

Layer 6 **khud hierarchy create nahi karta**. Ye existing Knowledge Forest ko use karta hai. Full data flow me Layer 5 ka output Knowledge Forest hota hai aur Layer 6 usme query → embedding → seed finding → traversal → Tree Knapsack karta hai. `filecite\turn2file5\L459-L480`

1. Layer 6 ko exact input kya milta hai?

Direct input

User:

```
"How does Round Robin improve fairness and what is its effect on response time?"
```

Report ke architecture summary ke according Layer 6 ka formal input:

```
User query
```

aur formal output:

```
RetrievalResult
```

`filecite\turn2file5\L483-L495`

Lekin internally Layer 6 ke paas sirf query string nahi hoti. Retrieval engine initialize hote waqt iske paas ye dependencies bhi hoti hain:

```
RetrievalEngine(
    forest,
    vector_index,
    cache_manager,
    config
)
```

`filecite\turn2file0\L22-L29`

Iska matlab Layer 6 ko practically ye milta hai:

```
| Input/Dependency | Example |
|---|---|
| User Query | "How does Round Robin improve fairness..." |
| Knowledge Forest | OS → Scheduling → Round Robin |
| Vector Index | Node embeddings |
| Cache Manager | Previous query/result cache |
| Config | seed_k, token budget, etc. |
```

Hamare example ka exact initial input

```
{
  "query": "How does Round Robin improve fairness and what is its effect on response time?"
}
```

Aur system ke paas already:

```
Knowledge Forest:
OS
├── CPU Scheduling
│   └── Round Robin

Semantic relations:
Round Robin → Fairness
Round Robin → Response Time
```

2. Input ka format kya hai?

User-facing input format

Primary input simple string hai:

```
query: str
```

Example:

```
query = "How does Round Robin improve fairness and what is its effect on response time?"
```

Report explicitly Layer 6 input ko **User query string** define karta hai. [filecite\turn2file2\L219-L224](#)

Lekin internal processing ke baad format change hota rehta hai

Ye bahut important hai.

Stage A

```
String
```

↓

Stage B: Query analysis

```
{
  intent: ...,
  complexity: ...
}
```

↓

Stage C: Embedding


```
Query Vector
[
  0.12,
  -0.48,
  0.73,
  ...
]
```

↓

Stage D: Vector search

```
Seed Nodes
[
  Round Robin,
  Fairness,
  Response Time,
  ...
]
```

↓

Stage E: Traversal

```
Candidate Nodes = set(...)
```

↓

Stage F: Context selection

```
Selected Nodes = list(...)
```

↓

Final

```
RetrievalResult
```

3. Layer 6 internally kya-kya karta hai?

Ab hum actual execution ko step-by-step dekhte hain.

STEP 0 — Cache Check

Sabse pehle:

```
cache_key = hash_query(query)
cached = self.cache.get_query_result(cache_key)

if cached:
    return cached
```

filecite\turn2file0\L30-L35

Hamare example me

System hash banayega:

```
Query:
"How does Round Robin improve fairness and what is its effect on response time?"

↓ hash

query_hash = X8A29...
```

Phir Redis/cache check hoga:

```
Kya exact same query recently process hui hai?
```

Case 1: Cache HIT

Agar answer ka retrieval result already stored hai:

```
Cache
  ↓
RetrievalResult mil gaya
  ↓
Layer 6 ka expensive processing skip
  ↓
Direct Layer 7
```

Case 2: Cache MISS

Hamare detailed example me maan lo:

```
CACHE MISS
```

Ab normal retrieval pipeline chalegi.

STEP 1 — Query Intent aur Complexity Analysis

Code flow:

```
intent = detect_intent(query)
complexity = classify_complexity(intent)
```

```
□filecite□turn2file0□L37-L40□
```

Hamari query:

"How does Round Robin improve fairness and what is its effect on response time?"

System isme dekhta hai:

```
"How does"    → explanatory / causal
"improve"     → effect
"fairness"    → related concept
"effect on"   → causal relationship
"and"         → multiple concepts
```

Report me query-complexity logic ke features include:

- number of entities
- comparison words
- causal words
- estimated reasoning hops

filecite\turn2file8\L717-L727

Isliye hamari query ko likely:

Intent: EXPLANATION / MULTI-HOP
Complexity: COMPLEX

classify kiya jayega.

Reasoning hops heuristic

Report ke example logic me:

- how, why, explain → extra hop
- affect, impact, effect, influence → extra hop
- conjunctions → multi-concept indication

filecite\turn2file9\L769-L790

Hamari query me ye signals present hain.

STEP 2 — Query Embedding

Ab:

```
query_emb = embed_query(query)
```

Report me specifically asymmetric embedding model use karne ka mention hai. filecite\turn2file0\L41-L43

Conceptually:

```
"How does Round Robin improve fairness..."
      ↓
    Embedding Model
      ↓
    Query Semantic Vector
```

Example:

```
Q =
[0.18, -0.42, 0.77, 0.09, ...]
```

Ye exact numbers report me defined nahi hain; ye sirf format samjhane ke liye illustrative example hai.

STEP 3 — Seed Nodes dhundhna

Ab vector search:

```
seed_nodes = self.vector_index.search(  
    query_emb,  
    k=5  
)
```

Default report configuration me seed count 5 diya gaya hai. `filecite\turn2file0\L41-L43`

Hamare example me vector search ka hypothetical result:

```
Seed 1 → Round Robin  
Seed 2 → Fairness  
Seed 3 → Response Time  
Seed 4 → CPU Scheduling  
Seed 5 → Time Quantum
```

Important:

Seed node final answer context nahi hota.

Seed ka matlab hai:

“Forest me yahan se exploration start karo.”

Yahi traditional flat RAG aur SHIA-RAG ka important difference hai.

Traditional RAG:

```
Top-k chunks = final retrieved context
```

SHIA-RAG:

```
Top-k seed nodes = exploration starting points
```

Uske baad hierarchy traversal hota hai. `filecite\turn2file1\L127-L156`

STEP 4 — Adaptive Depth decide karna

Hamari query:

```
Complexity = COMPLEX
```

Report ka configuration:

```
{  
  'COMPLEX': {  
    'max_depth': 3,  
    'include_parents': True,  
    'include_children': True,  
    'include_crosslinks': True  
  }  
}
```

`filecite\turn2file0\L45-L51`

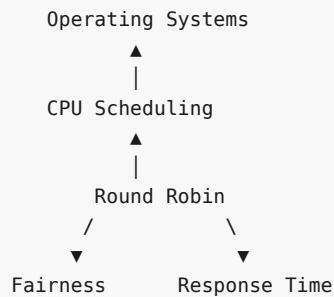
So hamari query ke liye system:

Maximum depth = 3

Parents = YES
Children = YES
Cross-links = YES

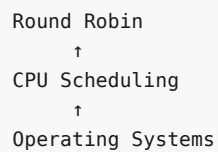
Ab hamare forest me traversal kaise hoga?

Seed: Round Robin



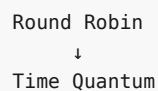
Because query COMPLEX hai:

Parent direction:



Child direction:

Agar Round Robin ke children hain to:



Cross-link direction:



Candidate set banega

Code:

```

candidate_nodes = set()

for seed in seed_nodes:
    candidate_nodes |= self.traverse(seed, traversal)
  
```

filecite\turn2file0\L53-L56

Example:

```
Seed 1 traversal:
{Round Robin, CPU Scheduling, OS, Fairness, Response Time}

Seed 2 traversal:
{Fairness, Round Robin}

Seed 3 traversal:
{Response Time, Round Robin}

Seed 4 traversal:
{CPU Scheduling, Round Robin, FCFS, SJF}

-----

Final Candidate Set:
{
  OS,
  CPU Scheduling,
  Round Robin,
  Fairness,
  Response Time,
  FCFS,
  SJF,
  ...
}
```

Duplicate nodes automatically set union se remove ho jate hain.

STEP 5 — Multi-Tree Knapsack

Ye Layer 6 ka sabse important algorithm hai

Ab problem ye hai:

Candidate nodes bahut mil gaye.

Lekin Layer 7 ko hum sab nahi bhej sakte.

Maan lo:

```
Total candidate information = 15,000 tokens
Available LLM budget = 4,000 tokens
```

To question:

Kaunse nodes select karein?

Traditional approach:

```
Top similarity nodes select kar lo
```

SHIA-RAG:

```
Tree Knapsack optimization
```

Simple intuition

Maan lo candidates:

```
| Node | Utility | Token Cost |
|---|---|---|
| Operating Systems | 0.3 | 200 |
| CPU Scheduling | 0.7 | 300 |
| Round Robin | 0.98 | 500 |
| Fairness | 0.92 | 400 |
| Response Time | 0.95 | 450 |
| FCFS | 0.45 | 500 |
| SJF | 0.40 | 450 |
```

Token budget:

1500 tokens

Hum random highest score nodes nahi lenge.

Tree structure bhi respect karni hai.

Report Tree Knapsack ko hierarchical precedence constraints ke saath context selection define karta hai.

□filecite□turn2file10□L836-L842□

Conceptually:

```
Round Robin choose
+
CPU Scheduling context
+
Fairness
+
Response Time
```

lekin irrelevant:

FCFS
SJF

skip ho sakte hain.

Multi-Tree kyun?

Ek query multiple topics ko touch kar sakti hai.

Example:

"Compare Round Robin scheduling with TCP congestion control"

Ab seeds different trees me ho sakte hain:

Tree A → Operating Systems
Tree B → Computer Networks

Isliye algorithm:

Step 1

Candidates ko root ke basis par group karo.

Step 2

Har tree ko proportional token budget do.

Step 3

Har tree par Tree Knapsack run karo.

Step 4

Results merge karo.

Step 5

Total token budget respect karo.

Exactly ye multi-tree process report me defined hai. `filecite\turn2file1\L159-L178`

Hamare example me selected nodes

Suppose budget ke baad:

SELECTED:

1. CPU Scheduling
2. Round Robin
3. Fairness
4. Response Time
5. Time Quantum

Rejected:

FCFS
SJF
Operating Systems

Why?

Because current question specifically fairness aur response time ke baare me hai.

STEP 6 — Context Assembly

Ab selected nodes ko simply random order me concatenate nahi karna.

Code:

```
context_text = self.assemble_context(selected, intent)
```

`filecite\turn2file0\L64-L65`

Final Report ke pseudocode ke according context ko semantic order me assemble kiya jata hai—definitions first, then core content, confidence-based ordering. `filecite\turn2file1\L144-L148`

Hamare example me context:

```
[Context 1: CPU Scheduling]
CPU scheduling determines how the operating system selects
processes for CPU execution.

[Context 2: Round Robin]
Round Robin allocates CPU time in cyclic order using a fixed
time quantum...

[Context 3: Fairness]
Round Robin provides each ready process an opportunity to receive
CPU time...

[Context 4: Response Time]
The time quantum and cyclic scheduling behavior influence the
response time experienced by processes...
```

Ab Layer 7 ko clean structured evidence mil gaya.

5. Output exactly kya banta hai?

Layer 6 ka output:

```
RetrievalResult
```

Report me exact structure:

```
@dataclass
class RetrievalResult:
    selected_nodes: list
    context_text: str
    traversal_path_edges: list
    query_complexity: str
    query_intent: str
    seed_node_ids: list
    token_count: int
```

`filecite\turn2file0\L11-L20`

Hamare example ka output

```

RetrievalResult(
  selected_nodes=[
    "CPU Scheduling",
    "Round Robin",
    "Fairness",
    "Response Time",
    "Time Quantum"
  ],

  context_text="""
CPU scheduling ...
Round Robin ...
Round Robin provides fairness ...
Round Robin affects response time ...
Time quantum influences ...
""",

  traversal_path_edges=[
    OS → CPU_Scheduling,
    CPU_Scheduling → Round_Robin,
    Round_Robin → Fairness,
    Round_Robin → Response_Time
  ],

  query_complexity="COMPLEX",

  query_intent="EXPLANATION",

  seed_node_ids=[
    "Round_Robin",
    "Fairness",
    "Response_Time",
    "CPU_Scheduling",
    "Time_Quantum"
  ],

  token_count=1850
)

```

Exact node names/token values example ke liye illustrative hain. Actual output forest contents aur retrieval scores par depend karega.

6. Output next layer ko kaise jata hai?

Ab:

```

Layer 6
  |
  | RetrievalResult
  ▼
Layer 7

```

Layer 7 ko raw document ya entire forest nahi milta.

Usko milta hai:

```
Query
+
Selected Nodes
+
Assembled Context
+
Traversal Information
+
Complexity / Intent Metadata
```

Overall pipeline explicitly:

```
Layer 6 → Selected context nodes
      ↓
Layer 7 → Build prompt → Invoke LLM
      ↓
Verify claims → Bind citations
```

filecite turn2file5 L473-L480

Important point

Layer 6:

“Kya evidence relevant hai?”

Layer 7:

“Is evidence ke basis par answer kya hai, aur kya answer evidence-supported hai?”

7. Ye Layer needed kyun hai?

Agar Layer 6 na ho

Tumhare paas complete Knowledge Forest hai:

10,000 nodes

User ne sirf ek query puchi:

Round Robin ka response time par kya effect hai?

LLM ko 10,000 nodes nahi bhej sakte.

Traditional RAG:

```
Query
  ↓
Top 5 chunks
  ↓
LLM
```

Problem:

- hierarchy ignore
- parent context missing ho sakta hai
- related concept side me miss ho sakta hai

- fixed retrieval depth
- irrelevant similar chunks aa sakte hain

SHIA-RAG Layer 6:

```

Query
↓
Relevant seeds
↓
Hierarchy-aware exploration
↓
Query-dependent depth
↓
Semantic cross-links
↓
Token-budget optimization
↓
Structured context

```

Isliye Layer 6 Knowledge Forest ko usable retrieval system me convert karta hai.

8. Is Layer me possible errors kya hain?

Ab tumhara important point.

Error 1 — Wrong query complexity

Example:

Query actually COMPLEX hai

Lekin classifier bolta hai:

SIMPLE

Then:

Depth = 1

Aur system required nodes tak nahi pahunchta.

Complete Report me insufficient depth ko explicit failure pattern ke roop me identify kiya gaya hai: query ko deeper traversal chahiye tha, lekin classifier ne shallow depth choose ki. `filecite[turn1file13L769-L777]`

Error 2 — Wrong seed nodes

Query:

"Round Robin and response time"

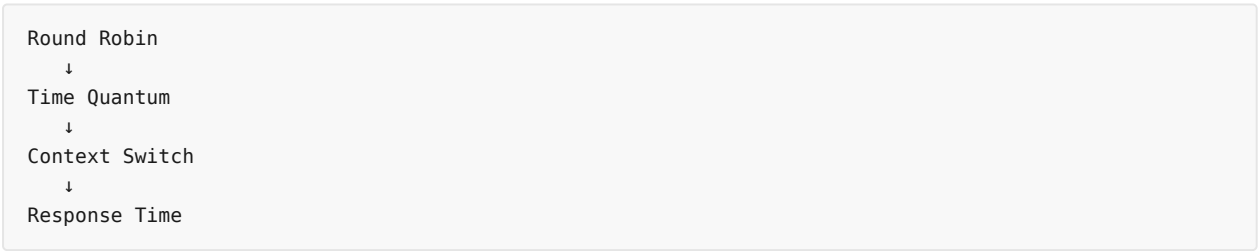
Vector DB galat concepts return kare:

Round Table
Robin bird
Response HTTP

To traversal galat region me start hoga.

Error 3 — Relevant node exists but traversal depth insufficient

Relevant path:



Required depth = 3

System:

depth = 1

Result:

Response Time miss

Error 4 — Cross-links disabled

Complex query ko semantic relation chahiye:

Round Robin → Fairness

Agar system complexity ko MODERATE classify kar de:

crosslinks = False

to fairness evidence miss ho sakta hai.

Error 5 — Token budget too small

Candidates:

Round Robin = 600 tokens
Fairness = 500
Response Time = 500
Required hierarchy context = 400

Total:

2000 tokens

Budget:

```
1000 tokens
```

Tree Knapsack ko important trade-off karna padega.

Potential risk:

```
Incomplete context
```

Error 6 — Multi-tree budget allocation issue

Agar:

```
Tree A = 90% relevant
Tree B = 10% relevant
```

Lekin budget proportional candidate count se allocate ho raha hai, Tree B ko unnecessary budget mil sakta hai.

Report me multi-tree grouping/allocation mechanism diya gaya hai, lekin relevance-aware budget reallocation ka detailed algorithm snippet me explicitly defined nahi hai. Is point par source ke beyond assume nahi karna chahiye.

Error 7 — Cache stale ho sakta hai

Same query ka old RetrievalResult cache me hai.

Lekin Layer 8 ne forest update kar diya.

Potentially:

```
New forest better hai
↓
Old cached retrieval aa gaya
```

Complete Report query result caching with TTL specify karta hai; query results ke liye 1-hour TTL shown hai.
[filecite]turn2file7[L608-L643]

9. Corrected Report ke according safeguards kya hain?

Safeguard 1 — Adaptive depth

Fixed depth use nahi hoti.

```
SIMPLE → depth 1
MODERATE → depth 2
COMPLEX → depth 3
```

Aur parents/children/cross-links selectively enable hote hain. [filecite]turn2file0[L45-L51]

Safeguard 2 — Multi-seed retrieval

System ek hi seed par depend nahi karta.

Default design:

```
k = 5 seed nodes
```

Agar ek seed weak hai to dusra seed relevant forest region tak le ja sakta hai. [filecite\turn2file0\L41-L43](#)

Safeguard 3 — Candidate set union

Multiple seeds same nodes retrieve kar sakte hain.

Set union:

```
candidate_nodes |= ...
```

Duplicate processing reduce karta hai. [filecite\turn2file0\L53-L56](#)

Safeguard 4 — Tree Knapsack

Context selection sirf similarity-based top-k nahi hai.

Hierarchy-aware precedence constraints maintain kiye jate hain.

Yani system semantic relevance aur structural context dono consider karta hai.

Safeguard 5 — Multi-tree processing

Multiple seed nodes different trees me hon to:

```
Group by root
↓
Per-tree optimization
↓
Merge
↓
Total budget trim
```

[filecite\turn2file1\L159-L178](#)

Safeguard 6 — Cache fallback

Agar Vector DB unavailable ho:

```
try:
    results = self.vector_search(query_embedding)
except VectorDBUnavailable:
    results = self.cache.get_recent_results(query_hash)
```

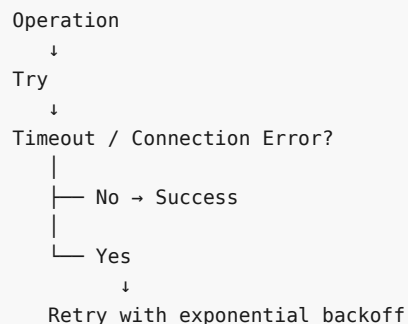
Yani vector retrieval fail hone par Redis/cache fallback specified hai. [filecite\turn2file7\L597-L603](#)

10. Exception handling kaise ho rahi hai?

Ab isko separate aur clearly samjhte hain.

A. Retry mechanism

Corrected report me generic retry decorator diya gaya hai:



Configuration:

```
max retries = 3
base delay = 1 second
max delay = 30 seconds
```

Delay:

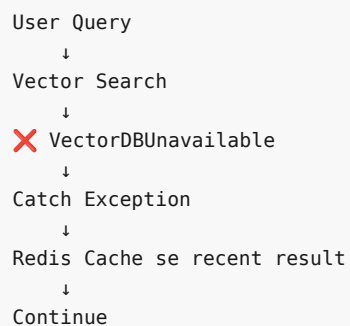
```
Attempt 1 failure → 1 sec
Attempt 2 failure → 2 sec
Attempt 3 failure → final raise
```

Report ka retry logic `TimeoutError` aur `ConnectionError` ke liye exponential backoff use karta hai.

📄filecite🔗turn2file3🔗L313-L334🔗

Layer 6 specific exception flow

Case 1 — Vector DB unavailable



Ye report me directly specified hai. 📄filecite🔗turn2file7🔗L597-L603🔗

Case 2 — Network/connection timeout

For operations covered by retry mechanism:


```
try operation
  ↓
TimeoutError / ConnectionError
  ↓
retry
  ↓
retry
  ↓
retry
  ↓
still failure?
  ↓
raise exception
```

filecite turn2file3 L319-L334

Case 3 — Cache unavailable

Important source-grounded point: Complete Report caching architecture describe karta hai, lekin retrieved Layer 6 snippets me cache server failure ka explicit Layer-6-specific fallback path define nahi hai.

Isliye main ye claim nahi karunga ki:

“Redis fail hone par automatically vector DB se continue hoga”

Not explicitly specified in the retrieved report sections.

Case 4 — No seed nodes found

Ye bhi important hai.

Maine jo Layer 6 specification retrieve ki hai usme:

```
zero seed nodes
```

ke liye explicit exception/fallback policy specified nahi mili.

Possible behavior logically design karna padega, **lekin source ke according exact policy “Not mentioned in the retrieved Layer 6 specification.”**

Ye future implementation gap ho sakta hai.

Complete Layer 6 — ek hi example ka full journey

Ab poor process ek saath dekho.

INPUT

```
"How does Round Robin improve fairness and what is its effect on response time?"
```

↓

0. CACHE

Cache HIT?
↓ No
Continue

↓

1. QUERY ANALYSIS

Intent:
EXPLANATION / MULTI-HOP

Complexity:
COMPLEX

↓

2. EMBEDDING

Query
↓
Embedding Model
↓
[0.18, -0.42, 0.77, ...]

↓

3. SEED SEARCH

Top 5 Seeds:

1. Round Robin
2. Fairness
3. Response Time
4. CPU Scheduling
5. Time Quantum

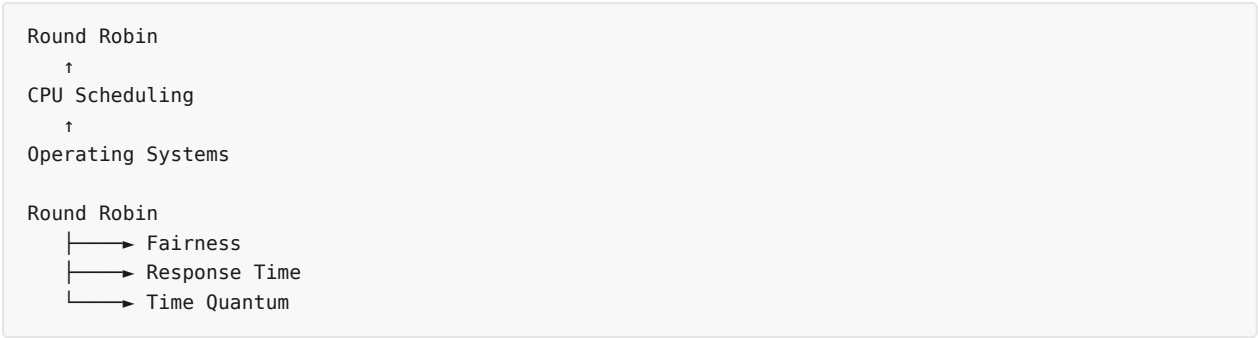
↓

4. ADAPTIVE TRAVERSAL

COMPLEX

Depth = 3
Parents = YES
Children = YES
Cross-links = YES

↓



↓

5. CANDIDATE NODES

```
{
  OS,
  CPU Scheduling,
  Round Robin,
  Fairness,
  Response Time,
  Time Quantum,
  FCFS,
  SJF,
  ...
}
```

↓

6. MULTI-TREE KNAPSACK

Token budget:

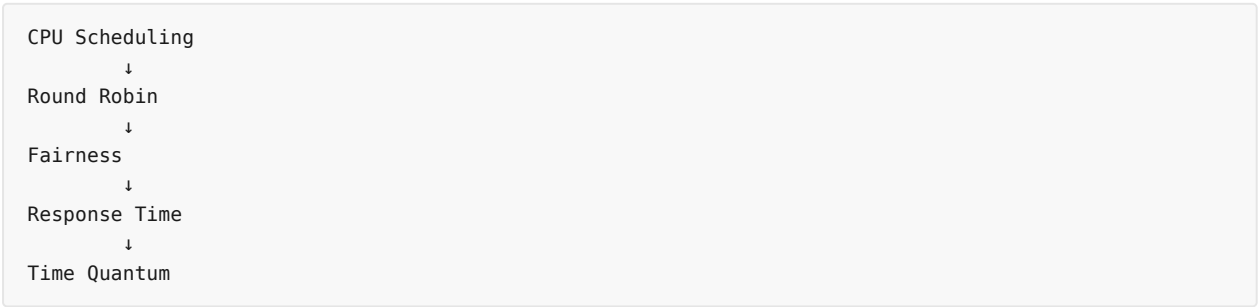
1500

Select:

- ✓ CPU Scheduling
- ✓ Round Robin
- ✓ Fairness
- ✓ Response Time
- ✓ Time Quantum
- x FCFS
- x SJF

↓

7. CONTEXT ASSEMBLY



↓

FINAL OUTPUT

```
RetrievalResult(  
    selected_nodes=[...],  
    context_text="...",  
    traversal_path_edges=[...],  
    query_complexity="COMPLEX",  
    query_intent="EXPLANATION",  
    seed_node_ids=[...],  
    token_count=...  
)
```

↓

LAYER 7

```
RetrievalResult  
    ↓  
Prompt Assembly  
    ↓  
LLM Generation  
    ↓  
Claim Verification  
    ↓  
Citation Binding  
    ↓  
Final Answer
```

Layer 6 ko ek line me yaad rakhna

Layer 6 user ke question ko answer nahi karta; wo Knowledge Forest ke andar query ke according intelligent search karke Layer 7 ke liye sabse relevant, structurally coherent aur token-budget-compliant evidence package banata hai.

Aur Layer 6 ke **4 core mechanisms** yaad rakho:

1. Seed Retrieval
2. Adaptive Depth Traversal
3. Tree / Multi-Tree Knapsack
4. Context Assembly

Ye Layer 6 ka complete conceptual flow hai. Iske baad next naturally **Layer 7** aata hai, jahan isi `RetrievalResult` ko lekar actual answer generate aur verify kiya jayega.